

Appunti di Algoritmi Distribuiti e Reti Complesse

Zbirciog Ionut Georgian

6 aprile 2026

Indice

1	Modellazione Del Sistema	3
1.1	Definizioni Elementari	3
1.2	Assiomi e Restrizioni	6
1.2.1	Assiomi	6
1.2.2	Resitrizioni	7
2	Broadcast	9
2.1	Algoritmo Flooding	10
2.1.1	Complessità Temporale	12
2.1.2	Stati ed Eventi	13
2.1.3	Lowerbound alla Message Complexity	14
3	Spanning Tree Construction	16
3.1	Single Source SHOUT Protocol	17
3.1.1	Terminazione	19
3.1.2	Correttezza	19
3.1.3	Message Complexity	20
3.2	Single Source SHOUT+ Protocol	22
3.3	Multiple Sources Spanning Tree Construction	24
4	Wake-Up Problem	25
4.1	Analisi dell'algoritmo WFlood	26
4.1.1	Correttezza	26
4.1.2	Message Complexity	26
4.1.3	Ideal Time Complexity	27
4.2	Lower Bound su Clique (Da FARE)	27
5	Labeled Hypercube	28
5.1	Broadcast su Labeled Hypercube	28
5.1.1	Costruzione Hypercube	29
5.2	Protocollo Hyperflood	31
6	Depth First Traversal (DFT) Protocol	34
6.1	Protocollo BACK	34
6.2	Protocollo BACK+	36
7	Tree Computations	38
7.1	Saturation Technique	38

7.2	Minimum Finding	39
7.3	Eccentricities & Center Finding	39
8	Leader Election	40
8.1	All The Way	41
8.1.1	Analisi	42
8.2	As Far	44
8.2.1	Analisi	46
8.3	Stage Technique	47
8.3.1	Analisi	49
8.4	Mesh Architecture	52
8.4.1	Protocollo	52
8.5	Randomized Algorithm	53
8.5.1	Analisi	53
9	Graph Coloring	56
9.1	Deterministic Distributed Graph Coloring	56
9.1.1	Basic Color Reduction Procedure - BCP	56
9.1.2	Khun-Wattenhofen Procedure	56
9.2	Randomized Distributed Graph Coloring	56
9.2.1	Rand- 2Δ	56
9.2.2	Rand- $\Delta+$	56

Capitolo 1

Modellazione Del Sistema

Definizione 1.0.1 (Sistema Distribuito). Un *sistema distribuito* è un insieme **entità** (dette anche **nodi**, **agenti**) che eseguono un algoritmo cooperando tra loro, comunicando e coordinandosi per raggiungere un obiettivo comune. Ogni nodo ha una propria memoria locale e può comunicare con altri nodi attraverso un canale di comunicazione. Internamente, ciascun nodo possiede un clock locale, che scandisce il tempo per le comunicazioni e le operazioni. I vari clock dei nodi non sono necessariamente sincronizzati tra loro, e possono procedere a velocità diverse. Inoltre, i nodi possono subire guasti, come crash o malfunzionamenti, che possono influenzare il comportamento del sistema.

Tale ambiente distribuito viene modellato mediante grafo $G = (V, E)$, dove V ($|V| = n$) è l'insieme dei vertici (nodi) e E ($|E| = m$) è l'insieme degli archi (interconnessioni). In particolar modo, data una rete costituita da agenti e interconnessioni tra di essi, il grafo sottostante ha un vertice per ciascun agente del sistema e un arco tra due nodi per ogni interconnessione tra i corrispettivi agenti.

1.1 Definizioni Elementari

Definizione 1.1.1 (Entità). Un'ambiente distribuito è costituito da un insieme $\mathcal{E}$ di entità. Ogni nodo $x \in \mathcal{E}$ della rete, ha delle capacità computazionali, le quali sono definite come un insieme di operazioni possibili, tra le quali

- **Comunicazione**: capacità di inviare e ricevere messaggi ad altri nodi della rete.
- **Local storage & processing**: capacità di memorizzare e elaborare informazioni localmente.
- **Clock**: capacità di tenere traccia del tempo, impostando un clock locale che scandisce il tempo per le comunicazioni e le operazioni. I vari clock dei nodi non sono necessariamente sincronizzati tra loro, e possono procedere a velocità diverse.
- **Modificare registri locali**: capacità di modificare i propri **registri locali**, che rappresentano lo stato interno del nodo.

Definizione 1.1.2 (Stato). Definiamo Σ come l'insieme di tutti gli stati che può assumere un nodo $x \in \mathcal{E}$. L'insieme degli stati è definito a priori e deve essere un insieme finito. Ad

esempio,

$$\Sigma = \{\text{idle, computing, waiting, } \dots\}$$

È importante notare che ad ogni istante di tempo t , un nodo x si trova in uno stato specifico $s \in \Sigma$. Lo stato di un nodo può cambiare nel tempo a seguito di operazioni computazionali o comunicazioni con altri nodi. Non esistono stati intermedi o transitori: un nodo è sempre in uno stato ben definito, e il passaggio da uno stato all'altro avviene in modo discreto, senza stati intermedi.

Definizione 1.1.3 (Evento). In un ambiente distribuito, il comportamento di ogni entità $x \in \mathcal{E}$ è **reattivo**, ovvero è indotto a seguito del verificarsi di **eventi**. In assenza di stimoli, un'entità rimane inerte.

In generale, gli eventi possono essere di vario tipo, come ad esempio:

- **Ricezione di un messaggio**: quando un nodo riceve un messaggio da un altro nodo, questo evento può indurre il nodo a cambiare stato o a eseguire determinate operazioni.
- **Tick del clock**: quando il clock locale di un nodo raggiunge un certo valore, questo evento può indurre il nodo a eseguire determinate operazioni o a cambiare stato.
- **Evento spontaneo**: un nodo può essere indotto a eseguire determinate operazioni o a cambiare stato in modo spontaneo.

Definizione 1.1.4 (Azione). Un' **azione** è una trasformazione che può essere eseguita da un nodo $x \in \mathcal{E}$ in risposta a un evento. Ogni azione modifica lo stato di un nodo o invia un messaggio ad un altro nodo. Formalmente, è una funzione **deterministica** che, fissato un istante di tempo t , mappa tutte le coppie (stato, evento) in un'azione specifica.

$$f : \Sigma \times \hat{E} \rightarrow \mathcal{A}$$

dove $\mathcal{A}$ è l'insieme di tutte le **sequenze di task** che possono essere eseguite, e $\hat{E}$ è l'insieme di tutti gli eventi che possono verificarsi in un ambiente distribuito.

In generale, un'azione può consistere in:

- **computare** un algoritmo;
- **inviare** messaggi ad altri nodi;
- **modificare** lo stato interno del nodo.

In questo modello, ogni azione è **atomica** e **finita**:

- **Atomica**: una volta avviata, un'azione non può essere interrotta;
- **Finita**: un'azione termina sempre in tempo finito.

Si osserva che un'entità in un certo stato potrebbe non reagire a un evento: in questo caso si assume che l'entità abbia eseguito l'azione *nil* (azione nulla).

Definizione 1.1.5 (Comportamento di un'entità). Le associazioni (stato, evento) $\rightarrow$ azione, per ogni nodo $x \in \mathcal{E}$, sono dette **regole**. Definiamo il **comportamento** di un'entità x come l'insieme di tutte le regole che governano le azioni che il nodo esegue in risposta

a eventi specifici, a seconda dello stato in cui si trova. Diremo che il comportamento di un'entità è

- **deterministico** se, per ogni coppia (stato, evento), esiste una regola che associa un'unica azione,
- **completo** se per ogni coppia (stato, evento) esiste almeno una regola che associa un'azione.

Il **comportamento** di un nodo $x \in \mathcal{E}$ è descrivibile mediante l'insieme $B(x)$ che deve essere **non-ambiguo** e **completo**, ossia:

$$\forall (s, e) \in \Sigma \times \hat{E}, \exists! a \in \mathcal{A} : f(s, e) = a$$

Definizione 1.1.6 (Comportamento del Protocollo). Definiamo il **comportamento del protocollo** come l'insieme di tutti i comportamenti dei nodi che partecipano al sistema distribuito. Formalmente è descritto come:

$$B(\mathcal{E}) = \{B(x) : x \in \mathcal{E}\}$$

Un sistema si dice **simmetrico (o omogeneo)** se tutti i nodi condividono lo stesso comportamento, ovvero:

$$B(x) \equiv B(y) \quad \forall x, y \in \mathcal{E}$$

Si nota che un sistema simmetrico è caratterizzato da un protocollo unico che viene eseguito da tutti i nodi. Questo significa che durante l'esecuzione i nodi possono effettuare azioni diverse ma il protocollo deve essere lo stesso. In un sistema asimmetrico (o eterogeneo), l'azione del singolo nodo dipende dalla sua etichetta.

In un sistema **asimmetrico (o eterogeneo)**, il comportamento del singolo nodo dipende dalla sua identificazione o dal suo ruolo. Tuttavia, qualsiasi sistema asimmetrico può essere trasformato in uno simmetrico riscrivendo l'insieme di regole in modo che l'azione eseguita dipenda dalla tipologia del nodo.

Esempio: Consideriamo un sistema con due tipologie di entità: server e client, ognuna con un insieme di regole differenti. È possibile unificare i comportamenti creando un nuovo insieme di regole simmetrico in cui ogni nodo esegue la stessa logica, ma l'azione concreta varia in base al ruolo dell'entità. In questo modo, il protocollo rimane unico e simmetrico per tutti i nodi.

Definizione 1.1.7 (Comunicazione). Le entità in un sistema distribuito interagiscono attraverso lo **scambio di messaggi** (message passing). Un messaggio è una **sequenza finita di bit** che transita su canali di comunicazione tra nodi. La topologia di comunicazione è rappresentata da un grafo diretto $G = (V, E)$ ($V = \mathcal{E}$), dove un arco $(x, y) \in E$ indica che il nodo x può inviare messaggi al nodo y . Non tutti i nodi possono comunicare direttamente: la capacità di comunicazione tra due nodi è determinata dalla struttura della rete sottostante.

Si osserva che i nodi *non hanno una visione globale* del sistema (ossia non conoscono la topologia del grafo della rete), ma solo una **visione locale**. Formalmente, $\forall x \in V$, possiamo definire gli insiemi:

- $N_{\text{out}}(x) = \{y \in V : (x, y) \in E\} \subset V$, l'insieme dei nodi a cui x può inviare messaggi (*archi uscenti*);
- $N_{\text{in}}(x) = \{y \in V : (y, x) \in E\} \subset V$, l'insieme dei nodi da cui x può ricevere messaggi (*archi entranti*).

Dunque, definiamo il **vicinato** di un nodo x come

$$N(x) = N_{\text{out}}(x) \cup N_{\text{in}}(x)$$

Definizione 1.1.8 (Tempo di Terminazione). Un protocollo si dice **terminante** se, a partire da qualsiasi configurazione iniziale, raggiunge una configurazione finale in un tempo finito. Formalmente, esiste un tempo finito t tale che, per ogni nodo $x \in V$, il nodo raggiunge uno stato *quiescente* (ossia, uno stato in cui non esegue più azioni) entro t .

Definizione 1.1.9 (Tempo di Completamento). Un protocollo si dice **completante** se, a partire da qualsiasi configurazione iniziale, raggiunge una configurazione finale **accettabile** in un tempo finito. In altre parole, esiste un istante di tempo finito t , per cui il task è risolto, ossia, ogni nodo $x \in V$ si trova in uno stato accettabile entro t .

In generale, un protocollo terminante non è necessariamente completante, e viceversa.

1.2 Assiomi e Restrizioni

La definizione di sistema distribuito è accompagnata da 2 *assiomi fondamentali*. Qualsiasi assunzione aggiuntiva che si vuole fare su un sistema distribuito sarà chiamata *restrizione*

1.2.1 Assiomi

Finite Transmission Delays In assenza di perdite di messaggi, si assume che il tempo di trasmissione di un messaggio tra due nodi sia finito. Ciò implica che, se un nodo invia un messaggio a un altro nodo, il messaggio arriverà entro un tempo finito, anche se questo tempo può essere arbitrariamente lungo, di conseguenza, il sistema non è a conoscenza di questo tempo di trasmissione.

Formalmente, $\forall$ messaggio m inviato da un nodo x a un nodo y , esiste un tempo finito $\tau_{xy} \in \mathbb{R}^+$ tale che m arriva a y entro τ_{xy} .

Local Orientation Questo assioma stabilisce che ogni nodo distingue localmente i suoi vicini, ossia, distingue tra gli archi entranti e quelli uscenti. Un'entità è sempre in grado di inviare un messaggio solamente ad uno specifico e singolo vicino uscente. Di conseguenza, è in grado di distinguere quale dei suoi vicini gli ha inviato un messaggio. Formalmente, ogni nodo x ha una funzione locale iniettiva di orientamento, che assegna un'etichetta univoca a ciascun arco:

$$\lambda_x : E_x \rightarrow L$$

dove $E_x = \{(x, y) \in E\}$ è l'insieme degli archi incidenti su x e L è un insieme di etichette.

Possiamo quindi discutere di grafi etichettati (G, λ) ($\lambda = \{\lambda_x : x \in V\}$), dove ogni arco (**non nodo**) (x, y) ha un'etichetta **locale** $\lambda_x(x, y)$ che identifica univocamente

il vicino y per il nodo x . Allo stesso modo, l'arco (y, x) avrà un'etichetta $\lambda_y(y, x)$ che identifica x per il nodo y .



Figura 1.1: Ciascun arco ha due etichette.

In generale, $\lambda_x(x, y) \neq \lambda_y(x, y)$.

1.2.2 Resitrazioni

Spesso i protocolli vengono definiti e studiati assumendo ulteriori restrizioni. Naturalmente un protocollo definito sotto un gran numero di restrizioni è generalmente debole, nel senso che, rimuovendo alcune di queste restrizioni, il protocollo potrebbe non risolvere il problema preso in analisi. Un protocollo definito su un sistema con poche restrizioni invece risulta più generale, e dunque più forte.

Si elencano alcune tipologie di restrizioni.

Message Ordering Si assume che i messaggi inviati da un nodo x a un nodo y arrivino in ordine FIFO (First In First Out) (**FIFO queue**). Ciò significa che se x invia due messaggi m_1 e m_2 a y , con m_1 inviato prima di m_2 , allora y riceverà m_1 prima di m_2 . Questa restrizione semplifica la progettazione dei protocolli, poiché garantisce un ordine prevedibile dei messaggi tra coppie di nodi.

Bidirectional Links Si assume che le connessioni tra nodi sia *bidirezionale*, ossia, $\forall x, y \in \mathcal{E}$, se $(x, y) \in E$, allora anche $(y, x) \in E$ e inoltre $\lambda_x(x, y) = \lambda_y(y, x)$. Questa restrizione implica che se un nodo x può inviare messaggi a un nodo y , allora y può anche inviare messaggi a x , e che entrambi i nodi condividono la stessa etichetta per identificare la connessione tra di loro. Questa restrizione semplifica la progettazione dei protocolli, poiché garantisce una comunicazione simmetrica tra i nodi. Formalmente,

$$\forall x \in V, N_{\text{out}}(x) = N_{\text{in}}(x) = N(x) \quad \text{e} \quad \lambda_x(x, y) = \lambda_y(y, x) \quad \forall y \in N_{\text{out}}(x)$$

Reliability Restrictions Ci chiediamo: cosa succede al messaggio durante il tragitto? Possiamo fare delle assunzioni sulla disponibilità dei canali di comunicazione.

- **Guaranteed Delivery:** Ogni messaggio che viene inviato viene **ricevuto correttamente**, cioè non ci sono corruzioni di messaggi. Sotto questa restrizione, i protocolli non devono tenere conto di omissioni o corruzioni dei messaggi durante la trasmissione.
- **Partial Reliability:** Non si verificano guasti (ad esempio crash dei nodi, perdita di messaggi) dall'istante in cui si inizia ad osservare il sistema. Si osserva che, negli istanti precedenti all'inizio dell'osservazione, il sistema potrebbe aver subito guasti.

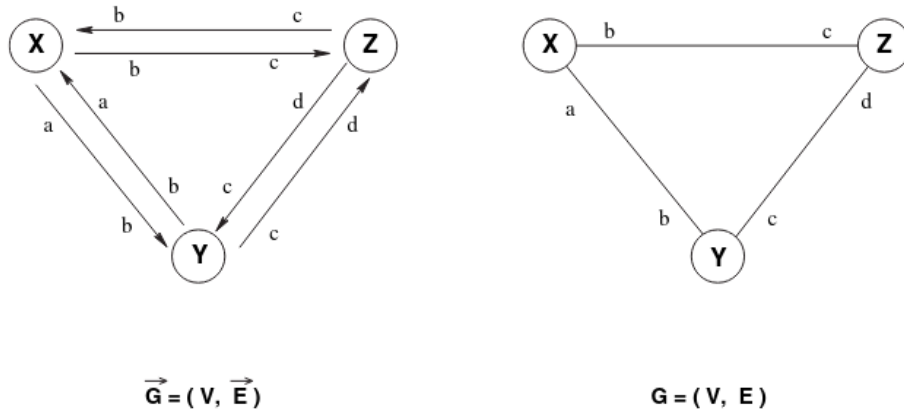


Figura 1.2: In una rete con archi bidirezionali, consideriamo il corrispondente grafo semplificato non diretto.

- **Total Reliability:** Non sono mai avvenuti fallimenti e mai avverranno in futuro.

Topological Restrictions In genere consistono in assunzioni sulla topologia del grafo della rete. Ad esempio assumere che il grafo sia **connesso**/**fortemente connesso**.

Knowledge Restrictions Assumiamo quali sono le informazioni che ogni nodo conosce riguardo la rete. Ad esempio in numero di nodi ($|V| = n$), oppure il numero di archi ($|E| = m$), il diametro della rete, l'etichettamento universale, etc... .

Time Restrictions Eccetto il tempo di trasmissione finito, il modello generale non fa assunzioni su tale quantità. Due possibili restrizioni riguardo i tempi di comunicazione sono le seguenti:

- **Bounded Communication Delay:** Esiste una costante Δ tale che, in assenza di guasti, il delay dei messaggi è al più Δ , in altre parole se un nodo x invia un messaggio m ad un nodo y , m arriverà ad y in al più Δ istanti di tempo.
- **Synchronized Clock:** Tutti i nodi condividono un clock globale sincronizzato, ossia, $\forall x, y \in \mathcal{E}$, $clock_x(t) = clock_y(t)$ per ogni istante di tempo t . Questa restrizione semplifica la progettazione dei protocolli, poiché consente ai nodi di coordinare le loro azioni basandosi su un riferimento temporale comune.

Cost & Complexity Measures In generale in un sistema distribuito non ha senso parlare di complessità temporale, poiché non è detto che i nodi condividano un clock globale. Per parlare di complessità temporale è necessario assumere la *Synchronized Clock* e la *Bounded Communication Delay*.

Parleremo spesso di **message complexity**, ovvero una misura che tiene conto del numero di messaggi scambiati durante l'esecuzione di un protocollo. Parleremo quindi di numero di bit scambiati, o di numero di messaggi scambiati.

Capitolo 2

Broadcast

Il primo esempio di comunicazione tra processi è il *broadcast*. Consideriamo un sistema distribuito dove un'entità ha una qualche informazione da condividere con tutte le altre entità. Il processo di inviare un messaggio a tutti gli altri processi è chiamato *broadcast*.

Sia $\mathcal{E}$ l'insieme di tutte le entità nel sistema distribuito e sia $G = (V, E)$ il grafo che rappresenta la rete di comunicazione, dove $V = \mathcal{E}$ e E è l'insieme degli archi che rappresentano le interconnessioni tra i nodi.

Prima di procedere con l'algoritmo di broadcast, è importante assumere alcune restrizioni sul sistema distribuito:

Bidirectional Links (BL): Consideriamo un grafo non diretto, dove ogni arco rappresenta una connessione bidirezionale tra due nodi.

Total Reliability (TR): Assumiamo che i messaggi inviati tra i nodi siano sempre consegnati senza errori o perdite.

Connectivity (CN): Il grafo G è connesso, il che significa che esiste un percorso tra ogni coppia di nodi nel grafo. Questa assunzione è fondamentale per garantire la risoluzione del problema di broadcast, poiché se il grafo non fosse connesso, alcuni nodi non potrebbero ricevere il messaggio.

Unique Initiator (UI): Supponiamo che ci sia un unico nodo che inizia il processo di broadcast, chiamato *initiator*. Questo nodo è responsabile di inviare il messaggio iniziale a tutti gli altri nodi.

Formalmente, il problema di broadcast può essere definito come segue: dato un grafo $G = (V, E)$ e un nodo $s \in V$ che è l'*initiator*, l'obiettivo è garantire che ogni nodo $v \in V$ riceva un messaggio da s .

Definiamo l'insieme $\Sigma = \{\text{initiator}, \text{idle}, \text{informed}\}$ come l'insieme di tutti gli stati che può assumere un nodo $x \in \mathcal{E}$.

Definiamo ora le configurazioni del sistema. Una configurazione C è una funzione che associa a ogni nodo $x \in \mathcal{E}$ uno stato in Σ . Inizialmente, la configurazione C_{init} è tale che esiste unico nodo $s \in V$ tale che $C_{init}(s) = \text{initiator}$ e $C_{init}(x) = \text{idle}$ per ogni nodo

$x \in V \setminus \{s\}$. Il numero di configurazioni iniziali possibili è

$$\binom{n}{1} = n,$$

dove n è il numero di nodi nel grafo. Questo perché possiamo scegliere qualsiasi nodo come initiator.

L'unica configurazione finale accettabile è quella in cui ogni nodo $x \in V$ è nello stato **informed**, ovvero

$$C_{final}(x) = \text{informed} \quad \forall x \in V.$$

Quindi, il problema del broadcast si formalizza mediante la quadrupla:

$$\langle \mathcal{R}, \Sigma, C_{init}, C_{final} \rangle,$$

dove $\mathcal{R} = \{\text{BL}, \text{TR}, \text{CN}, \text{UI}\}$ è l'insieme delle restrizioni, Σ è l'insieme degli stati, C_{init} è la configurazione iniziale e C_{final} è la configurazione finale desiderata.

2.1 Algoritmo Flooding

L'idea generale dell'algoritmo di broadcast è quella di utilizzare un approccio di flooding, dove ogni nodo che riceve il messaggio lo inoltra a tutti i suoi vicini. L'algoritmo può essere descritto come segue:

Algorithm 1 Algoritmo di Broadcast (Flooding)

```

if  $C(s) = \text{initiator}$  then
     $C(s) \leftarrow \text{informed}$ 
    for all  $v \in N(s)$  do
        send( $m, v$ )
    end for
end if

if  $C(x) = \text{idle}$  then
     $m \leftarrow \text{receive}()$ 
     $C(x) \leftarrow \text{informed}$ 
    for all  $v \in N(x) \setminus \{\text{sender}\}$  do
        send( $m, v$ )
    end for
end if

if  $C(x) = \text{informed}$  then
    do_nothing()
end if

```

Descritto in questa maniera, l'algoritmo di flooding risolve il task nella maniera corretta, e dimostreremo inoltre che termina in un tempo finito. Tuttavia, è importante notare che questo algoritmo non è efficiente in termini di complessità di messaggi, poiché ogni nodo

potrebbe ricevere lo stesso messaggio più volte da diversi vicini, portando a un numero esponenziale di messaggi inviati.

Osservazione. Se non considerassimo l'insieme degli stati $\Sigma = \{\text{initiator}, \text{idle}\}$ e non avessimo la condizione che un nodo non inoltra il messaggio se è già informato, l'algoritmo di flooding potrebbe non terminare, poiché i nodi continuerebbero a inoltrare il messaggio indefinitamente.

Lemma 2.1.1. *L'algoritmo di flooding termina in un tempo finito. Sia $\text{diam}(G)$ il diametro del grafo G , ovvero la massima distanza tra due nodi qualsiasi nel grafo.*

Esiste un tempo massimo τ tale che tutti i nodi x a distanza h da s ricevano il messaggio m entro il tempo τ .

$\forall h \leq \text{diam}(G), \exists \tau \in \mathbb{R}^+ : \forall x \in V \mid \text{dist}(x, s) = h \implies C_{\text{final}}(x) = \text{informed} \wedge t(x) \leq \tau.$

Dimostrazione. Dimostriamo il lemma per induzione su h .

Passo Base $h = 1$ Sfruttando la condizione di Total Reliability, ogni nodo v adiacente a s riceve il messaggio m entro un tempo finito τ_1 . Quindi, per ogni nodo v tale che $\text{dist}(v, s) = 1$, abbiamo $C_f(v) = \text{informed}$ e $t(v) \leq \tau_1$.

Passo Induttivo $h + 1$ Sfruttando la condizione di Connectivity, per ogni nodo $x \in V \setminus \{s\} : d(s, x) = h + 1$.

Questo implica che esiste un shortest path P da $s \rightarrow x$ di lunghezza $h + 1$. Sia y il nodo adiacente a x su questo percorso, quindi $\text{dist}(y, s) = h$. Per ipotesi induttiva, esiste un tempo finito τ_h tale che $C_{\text{final}}(y) = \text{informed}$ e $t(y) \leq \tau_h$.

Sfruttando nuovamente la condizione di Total Reliability, il nodo x riceve il messaggio da y entro un tempo finito τ_{h+1} , quindi $C_{\text{final}}(x) = \text{informed}$ e $t(x) \leq \tau_{h+1}$.

□

Lemma 2.1.2 (Message Complexity). *L'algoritmo di flooding ha una message complexity*

$$M[\text{Flooding}] = \theta(|E|) = \theta(m).$$

Dimostrazione. Dimostriamo il lemma utilizzando due metodi differenti. Nel primo metodo facciamo un'analisi diretta del numero di messaggi inviati, mentre nel secondo metodo utilizziamo un'analisi basata sui nodi e sui loro vicini.

Metodo I: L'idea è di contare il numero di messaggi che attraversano un arco. Se $\text{state}(s) = \text{initiator}$, il numero di messaggi che passano attraverso i suoi archi incidenti è esattamente 1, ossia solo il messaggio m da inoltrare. Consideriamo adesso due nodi $x \neq s$ e $y \neq s$ adiacenti e contiamo il numero di messaggi che passano attraverso l'arco (x, y) . Siano t_x e t_y gli istanti di tempo in cui x e y ricevono il messaggio m , e consideriamo il caso in cui $t_x < t_y$. In questo caso, x inoltra il messaggio m a y . Tuttavia, al tempo t_y , y riceve una copia del messaggio da un nodo $z \neq x$ un'istante prima di riceverlo da x . In questo caso, y inoltra il messaggio a x . Quindi, il numero massimo di messaggi che passano attraverso l'arco (x, y) è 2. Quindi ogni arco può essere attraversato da al massimo 2 messaggi, dimostrando che $M[\text{Flooding}] = \theta(2|E|) = \theta(m).$

Metodo II: Una dimostrazione più precisa può essere ottenuta considerando il numero di messaggi inviati da ogni nodo. Se $state(s) = \text{initiator}$, il numero di messaggi inviati è esattamente $d(s)$ dove $d(x) = N(x)$ è il grado di x . Consideriamo adesso un nodo $x \neq s$. Il numero di messaggi che invia è $d(x) - 1$, poiché non inoltra il messaggio al nodo da cui lo ha ricevuto. Quindi, il numero totale di messaggi inviati è

$$\begin{aligned} M[\text{Flooding}] &= d(s) + \sum_{x \neq s} (d(x) - 1) \\ &= \sum_{x \in V} d(x) - \sum_{x \in V \setminus \{s\}} (n - 1) \\ &= 2|E| - (n - 1) = 2m - n + 1 = \theta(m). \end{aligned}$$

□

Osservazione. La somma degli gradi di tutti i nodi in un grafo è pari a $2|E|$, poiché ogni arco contribuisce al grado di due nodi.

2.1.1 Complessità Temporale

In generale, non ha senso calcolare la complessità temporale di un algoritmo in un sistema asincrono a causa dell'impossibilità di poter predire il tempo di trasmissione dei messaggi. Tuttavia, se assumiamo un sistema sincrono, ossia assumiamo **clock sincronizzati** e **bounded delay**, possiamo calcolare la complessità temporale dell'algoritmo di flooding. Assumiamo che i messaggi vengano trasmessi con un delay di δ (normalizzando, $\delta = 1$). Chiaramente, il messaggio inviato dall'initiator s deve raggiungere ogni nodo x a distanza h da s entro un tempo finito, e il tempo necessario per raggiungere il nodo più lontano da s è dato **eccentricity** di s , che è definita come

$$ecc(s) = \max_{x \in V} dist(s, x).$$

Allora, nel caso peggiore, il tempo necessario per raggiungere ogni nodo è dato dal diametro del grafo $\text{diam}(G)$, che è definito come

$$\text{diam}(G) = ecc(s).$$

In conclusione, la complessità temporale *ideale* dell'algoritmo di flooding è

$$T[\text{Flooding}] = \max_{x \in V} dist(s, x) \leq \text{diam}(G) \leq n - 1 = O(n).$$

Si vuole dimostrare inoltre che tale protocollo sia ottimale in termini di complessità temporale, ossia che non esiste un protocollo di broadcast che possa raggiungere ogni nodo in un tempo minore del diametro del grafo. È necessario quindi dare una delimitazione inferiore alla complessità temporale di qualsiasi protocollo di broadcast. Si osserva che ciascuna entità x a distanza h da s non può ricevere il messaggio m prima di h unità di tempo, poiché il messaggio deve attraversare almeno h archi per raggiungere x . Quindi, la complessità temporale di qualsiasi protocollo di broadcast è limitata inferiormente da

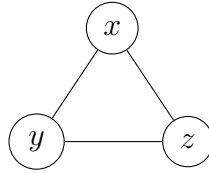
$$T[\text{Broadcast}] \geq \max_{x \in V} dist(s, x) = ecc(s) \leq \text{diam}(G).$$

Di conseguenza, l'algoritmo di flooding è ottimale in termini di complessità temporale, in particolare l'ideal time complexity è $T[\text{Flooding}] = \theta(\text{diam}(G))$.

2.1.2 Stati ed Eventi

Nel mondo centralizzato, dato uno stesso input ad un algoritmo deterministico, esso non solo genera lo stesso output, ma la sequenza di azioni che svolge per raggiungere tale output è sempre la stessa, ad ogni esecuzione. Questo dettaglio non vale nel mondo distribuito. Ad esempio nel protocollo flooding, dato che il ritardo dei messaggi varia in modo indeterminato, la sequenza di azioni verso l'unica configurazione finale varia ad ogni esecuzione, anche a fronte di una stessa configurazione iniziale (cioè esecuzioni dallo stesso iniziatore). Naturalmente, per la correttezza del protocollo, una qualsiasi sequenza di azioni anche se diversa converge verso la configurazione finale.

Consideriamo il seguente grafo G con $n = 3$ nodi e $m = 3$ archi, dove x è l'iniziatore:



Eseguendo una prima volta il protocollo, supponiamo che z riceva il messaggio da x prima di y .

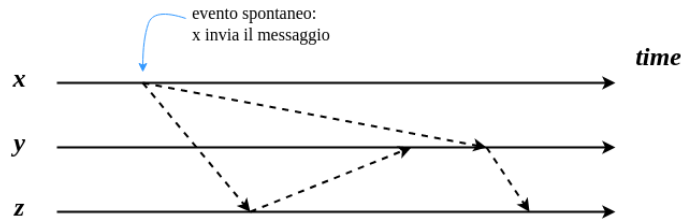
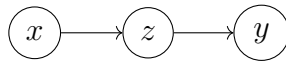


Figura 2.1: Sequenza di azioni del protocollo di flooding, con z che riceve il messaggio da x prima di y .

Tale esecuzione può essere rappresentata mediante il seguente grafo, dove ogni nodo rappresenta una configurazione del sistema e ogni arco rappresenta un evento (cioè, l'invio o la ricezione di un messaggio). In questo caso, la sequenza di azioni è $x \rightarrow z \rightarrow y$.

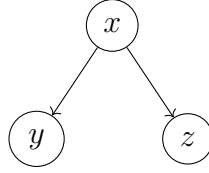


Eseguendo una seconda volta il protocollo, potrebbe capitare che i ritardi sugli archi (x, z) e (z, y) siano simili. Perciò, una possibile sequenza di azione potrebbe essere:



Figura 2.2: Esecuzione del protocollo con ritardi simili sugli archi (x, z) e (z, y) .

Tale esecuzione può essere rappresentata mediante il seguente grafo,



Dato che il grafo di comunicazione è sostanzialmente un albero ricoprente, ci sono quindi $O(n^n)$ possibili sequenze di trasmissioni (o **traiettorie**) che portano alla configurazione finale, anche a fronte di una stessa configurazione iniziale.

2.1.3 Lowerbound alla Message Complexity

Definizione 2.1.1 (Stato Locale Interno). Definiamo $\sigma(x, t)$ come lo stato locale interno del nodo x al tempo t . Di conseguenza

$$\Sigma(t) = \{\sigma(x, t) : x \in \mathcal{E}\}$$

è l'insieme di tutti gli stati locali dei nodi al tempo t .

Consideriamo adesso due ambienti (*environments*) E_1 e E_2 dove, lo stato interno di x al tempo t è lo stesso, ossia $\sigma_1(x, t) = \sigma_2(x, t)$. Allora x non è in grado di *distinguere* tra i due ambienti. Di conseguenza, se avviene lo stesso evento in entrambi gli ambienti, x esegue la stessa azione in entrambi gli ambienti.

Seguono due proprietà fondamentali che ci permettono di dimostrare un lowerbound alla message complexity di qualsiasi protocollo di broadcast.

Proprietà 1: Se x si trova in due ambienti E_1 e E_2 , e lo stato interno di x è lo stesso in entrambi gli ambienti al tempo t ($\sigma_1(x, t) = \sigma_2(x, t)$), allora x non è in grado di distinguere tra i due ambienti.

Proprietà 2: Se due nodi distinti x e y ad un certo istante t si trovano nello stesso stato interno ($\sigma(x, t) = \sigma(y, t)$), allora se vengono sollecitati dallo stesso evento, eseguiranno la stessa azione.

Teorema 2.1.3 (Lower Bound). *Sotto le ipotesi*

$$R = \{BL, TR, CN, UI\}$$

(ossia, *bidirectional links, total reliability, connectivity e unique initiator*), *qualsiasi protocollo di broadcast ha una message complexity* $M[\text{Broadcast}] = \Omega(m)$.

Dimostrazione. Supponiamo per assurdo che esista un protocollo di broadcast con una message complexity $M[\text{Broadcast}] = o(m)$ (per esempio $m - 1$ messaggi). Consideriamo un grafo $G = (V, E)$, e sia s l'initiator. Poiché il protocollo ha una message complexity sublineare, esiste almeno un arco $e = (x, y) \in E$ che non viene attraversato da alcun messaggio durante l'esecuzione del protocollo. Sia ora $G' = (V', E')$ costruito a partire da G rimuovendo l'arco e e aggiungendo un nuovo nodo z tale per cui $state(z) = \text{idle}$ connesso a x e y mediante due nuovi archi, ossia

$$V' = V \cup \{z\}, \quad E' = E \setminus \{e\} \cup \{(x, z), (y, z)\}.$$

Siano quindi E_1 e E_2 i due ambienti, dove in E_1 il protocollo viene eseguito su G , mentre in E_2 il protocollo viene eseguito su G' . Le etichette degli archi in G e G' sono le stesse,

quindi x e y non sono in grado di distinguere tra i due ambienti.

$$\lambda_x(x, y) = \lambda_x(x, z), \quad \lambda_y(y, x) = \lambda_y(y, z).$$

Allora sapendo che x e y non sono in grado di distinguere (**Proprietà 1**) tra i due

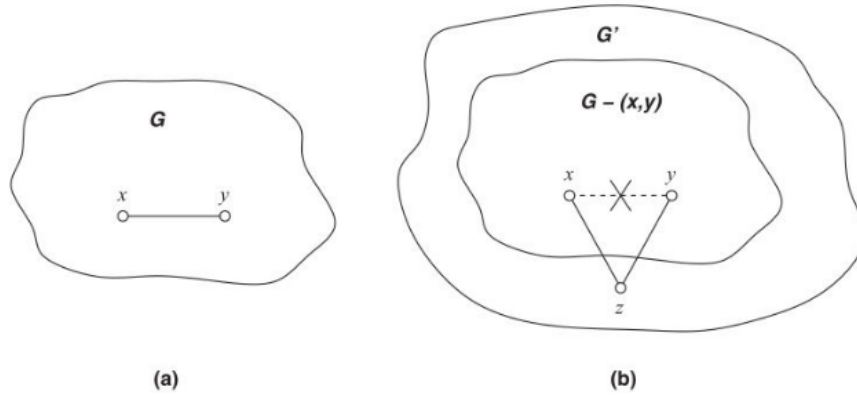


Figura 2.3: Costruzione del grafo G' a partire da G .

ambienti, se viene sollecitato lo stesso evento in entrambi gli ambienti, x e y eseguono la stessa azione (**Proprietà 2**). Poiché in E_1 non inoltrano il messaggio sul arco (x, y) , in E_2 non inoltrano il messaggio né su (x, z) né su (y, z) . Di conseguenza, il nodo z non riceve mai il messaggio (poiché è collegato solo a x e y), e quindi il protocollo di broadcast fallisce in E_2 , portando ad una contraddizione, ossia vengono inviati $o(m)$ messaggi e rimane un nodo non raggiungibile. $\square$

Osservazione. Se consideriamo una fase di preprocessing, ossia una fase iniziale in cui i nodi possono scambiarsi messaggi per costruire una mappa della rete (uno **spanning tree**), allora è possibile progettare un protocollo di broadcast con una message complexity $M[\text{Broadcast}] = O(n)$, che è ottimale in quanto ogni nodo deve ricevere almeno un messaggio. Tuttavia, in questo caso, la message complexity totale del protocollo (preprocessing + broadcast) è $M[\text{Total}] = O(n + m) = O(m)$, che è ancora ottimale in quanto ogni arco deve essere attraversato da almeno un messaggio durante la fase di preprocessing.

Capitolo 3

Spanning Tree Construction

Definizione 3.0.1 (Spanning Tree). Un albero ricoprente (Spanning Tree) di un grafo $G = (V, E)$ è un sottografo connesso e aciclico di G tale che $T = (V, E')$ dove $E' \subseteq E$. In altre parole è un albero che include tutti i nodi di V .

Definizione 3.0.2 (SPT Construction Distributed Problem). In un sistema distribuito, il problema di costruire un albero ricoprente (Spanning Tree) del grafo $G = (V, E)$ significa portare il sistema da uno stato iniziale, in cui ogni nodo è consapevole solo di se stesso e dei suoi vicini, a uno stato finale in cui tutti i nodi sono organizzati in una struttura ad albero che copre tutti i nodi del grafo. In altre parole,

Configurazione Iniziale: Ciascun nodo $x \in \mathcal{E} = V$ mantiene una struttura dati $\text{tree_neigh}(x) = \emptyset$. La possiamo immaginare come un vettore binario, dove ogni posizione rappresenta un nodo vicino ad x , e il valore 1 indica che l'arco tra x e quel nodo è parte dell'albero, mentre 0 indica che non lo è.

Configurazione Finale: $\forall x \in \mathcal{E} = V$,

$$\text{tree_neigh}(x) = \{\text{archi che appartengono allo ST}\},$$

e l'unione di tutti questi insiemi per forma un albero ricoprente del grafo G .

Il problema è formalizzato dalla tripla $P = \langle P_{init}, P_{final}, R \rangle$, dove:

$$P_{init} = \forall x \in V,$$

$$\begin{aligned} \text{tree_neigh}(x) &= \emptyset \\ \exists! s \in V, \text{state}(s) &= \text{initiator} \\ \forall x \in V \setminus \{s\}, \text{state}(x) &= \text{idle} \end{aligned}$$

$$P_{final} = \forall x \in V,$$

$$\begin{aligned} \text{tree_neigh}(x) &= \{y \in N(x) : (x, y) \in E'\} \\ \bigcup_{x \in V} \text{tree_neigh}(x) &= T(V, E') \text{ albero ricoprente del grafo } G \\ \forall x \in V, \text{state}(x) &= \text{done} \end{aligned}$$

$$R = \{\text{TR}, \text{BL}, \text{CN}, \text{UI}\}$$

Quindi, a differenza di un algoritmo centralizzato (Kruskal, Prim), la conoscenza dello ST è distribuita tra i nodi, e ogni nodo deve collaborare con i suoi vicini per costruire l'albero ricoprente. Non esiste quindi una visione globale dell'albero, ma ogni nodo deve basarsi solo sulle informazioni locali e sui messaggi ricevuti dai suoi vicini per decidere se includere o meno un arco nel suo insieme $\text{tree_neigh}(x)$.

3.1 Single Source SHOUT Protocol

L'idea dell'algoritmo è quella di chiedere ai nodi vicini se sono interessati a far parte dello ST, e se sì, includere l'arco che li collega al nodo che ha fatto la richiesta. In questo modo, partendo da un nodo iniziale (initiator), si costruisce gradualmente lo ST coinvolgendo i nodi vicini.

Definiamo $\Sigma = \{\text{initiator}, \text{idle}, \text{active}, \text{done}\}$ dove $S_{\text{init}} = \{\text{initiator}, \text{idle}\}$ l'insieme dei stati iniziali e $S_{\text{final}} = \{\text{done}\}$ l'insieme dei stati finali.

- Inizialmente, tutti i nodi tranne l'iniziatore s sono in stato **idle**, e s è in stato **initiator**. Spontaneamente, s si sveglia e invia i messaggi di domanda (Q) a tutti i suoi vicini, proponendo loro di essere connessi a lui nello ST (cioè essere suoi figli).
- Quando un nodo x nello stato **idle** riceve un messaggio Q, questo si attiva, sceglie come padre il **primo** nodo da cui ha ricevuto il messaggio e invia a tale nodo il messaggio YES.
- Successivamente, y invia a tutti i suoi vicini, tranne il padre, un messaggio Q per chiedere se vogliono essere suoi figli. Se un nodo z nello stato **active** riceve un messaggio Q da y , risponde con un messaggio NO perché ha già scelto un padre e non può accettare più di uno. Se invece z è nello stato **idle**, risponde con un messaggio YES e si attiva, scegliendo y come padre.
- Ogni nodo conta il numero di risposte ricevute da suoi vicini. Quando un nodo x ha ricevuto risposte da tutti i suoi vicini, passa allo stato **done** e non fa più nulla.

Per un nodo x , i vicini che rispondono con YES saranno inclusi nel suo insieme $\text{tree_neigh}(x)$ includendo anche il vicino che ha fatto la richiesta.

Possiamo quindi considerare il protocollo SHOUT come un broadcast con reply.

Algorithm 2 Algoritmo di SHOUT

```
if state(x) = initiator then spontaneously
    root  $\leftarrow$  true
    tree_neigh(x)  $\leftarrow$   $\emptyset$ 
    state(x)  $\leftarrow$  active
    counter  $\leftarrow$  0
    for all  $v \in N(x)$  do
        send(Q, v)
    end for
end if

if state(x) = idle then
    Q  $\leftarrow$  receive()
    tree_neigh(x)  $\leftarrow$  {sender}
    root  $\leftarrow$  false
    counter  $\leftarrow$  1
    send(YES, sender)
    if counter =  $|N(x)|$  then
        state(x)  $\leftarrow$  done
    else
        for all  $v \in N(x) \setminus \{\text{sender}\}$  do
            send(Q, v)
        end for
        state(x)  $\leftarrow$  active
    end if
end if

if state(x) = active then
    reply  $\leftarrow$  receive()
    if reply = YES then
        tree_neigh(x)  $\leftarrow$  tree_neigh(x)  $\cup$  {sender}
        counter  $\leftarrow$  counter + 1
        if counter =  $|N(x)|$  then
            state(x)  $\leftarrow$  done
        end if
    end if
end if

    if reply = NO then
        counter  $\leftarrow$  counter + 1
        if counter =  $|N(x)|$  then
            state(x)  $\leftarrow$  done
        end if
    end if

    if reply = Q then
        send(NO, sender)
    end if
end if
```

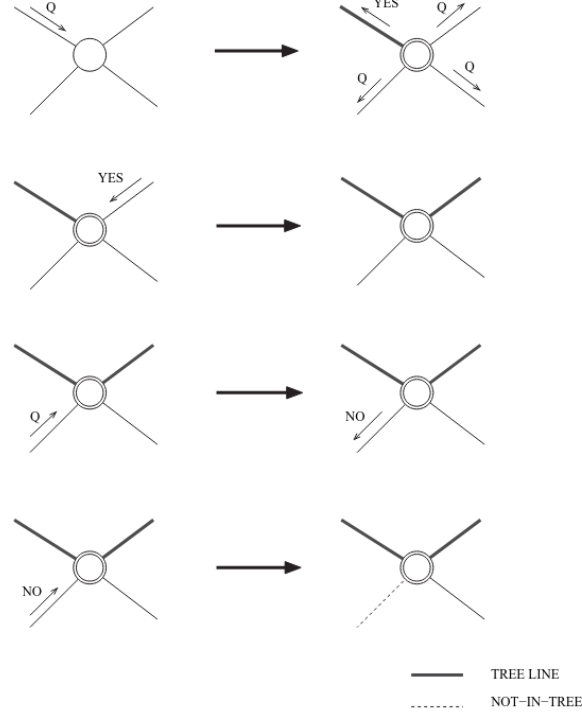


Figura 3.1: Insieme di regole del protocollo SHOUT.

3.1.1 Terminazione

Il protocollo SHOUT è molto simile al protocollo di broadcast con l'aggiunta dei messaggi di reply YES e NO. Poiché siamo sotto le assunzioni TR e CN, allora ogni nodo $x \neq s$ riceverà entro un tempo finito un messaggio Q da un vicino, e quindi si attiverà impostando il contatore a 1 (per la correttezza del protocollo di broadcast). Per ogni richiesta ricevuta, i nodi risponderanno con un messaggio YES o NO, di conseguenza, ogni nodo riceverà (per la TR e per la costruzione del protocollo, ogni nodo risponderà a tutte le richieste) un numero finito di risposte ($|N(x)|$), e quindi, entro un tempo finito, ogni nodo raggiungerà lo stato done, terminando localmente la propria esecuzione.

3.1.2 Correttezza

Per dimostrare la correttezza del protocollo SHOUT, dobbiamo mostrare che alla fine dell'esecuzione, l'insieme di archi scelti da tutti i nodi forma un albero ricoprente del grafo G .

Lemma 3.1.1 (Coerenza). *Siano x e y due nodi vicini in G . Se x ha come vicino y in ST , cioè $y \in \text{tree_neigh}(x)$, allora x è vicino di y in ST , cioè $x \in \text{tree_neigh}(y)$. Quindi*

$$\forall x, y \in V, y \in \text{tree_neigh}(x) \iff x \in \text{tree_neigh}(y).$$

Dimostrazione. Supponiamo che $y \in \text{tree_neigh}(x)$. Allora y ha risposto con un messaggio YES alla richiesta di x , e quindi y ha scelto x come padre. Ma se y ha risposto con YES a x , allora y era nello stato idle quando ha ricevuto la richiesta da x e per definizione del protocollo y inserisce x nel suo insieme $\text{tree_neigh}(y)$. Quindi $x \in \text{tree_neigh}(y)$. La dimostrazione dell'implicazione inversa è simile, con x e y scambiati. $\square$

Lemma 3.1.2 (Connettività/Spanning). *Il sottografo $T = (V, E')$ definito da $E' = \bigcup_{x \in V} \text{tree_neigh}(x)$ è connesso e include tutti i nodi di V . In altre parole, il sottografo indotto dagli archi su cui è viaggiato il messaggio YES è connesso e include tutti i nodi di V (è ricoprente e connesso).*

Dimostrazione. Vogliamo dimostrare che per ogni nodo $x \in V \setminus \{s\}$, se x ha risposto YES ad un nodo y , allora $x \in \text{tree_neigh}(y)$, dove quest'ultimo è connesso alla sorgente s tramite una sequenza di archi sui quali sono passati i messaggi YES, e tale sequenza formerà un cammino da $s \rightarrow x$.

Sia x un nodo qualsiasi in $V \setminus \{s\}$. Se x ha risposto YES ad un nodo y , allora $x \in \text{tree_neigh}(y)$, e dato che y deve aver inviato una proposta al nodo x , allora y si trova in **active** alla ricezione della risposta YES da x . Ma y invia una proposta al nodo x solo se:

- y è la sorgente s , e quindi è connesso a se stesso, oppure
- y ha ricevuto una proposta da un nodo z e ha risposto YES a z , e quindi y è connesso a z tramite un arco su cui è passato un messaggio YES, quindi

$$z \in \text{tree_neigh}(y) \text{ e } y \in \text{tree_neigh}(z) \quad \text{Lemma 3.1.1}$$

Iterando questo processo fino alla sorgente s , si ottiene un cammino che collega s a x tramite archi su cui sono passati messaggi YES, e quindi x è connesso a s .

□

Lemma 3.1.3 (Aciclicità). *Il sottografo $T = (V, E')$ definito da*

$$E' = \bigcup_{x \in V} \text{tree_neigh}(x) \text{ è aciclico.}$$

Dimostrazione. Supponiamo per assurdo che T contenga un ciclo. Allora esiste una sequenza di nodi $v_1, v_2, \dots, v_k$ con $k \geq 3$ tale che $(v_i, v_{i+1}) \in E'$ (ossia v_i è padre di v_{i+1}) per ogni $i = 1, 2, \dots, k-1$ e $(v_k, v_1) \in E'$.

Poichè ogni nodo, eccetto la sorgente invia **un solo** messaggio YES, allora il ciclo $v_1, v_2, \dots, v_k$ non contiene la sorgente s . Per il Lemma 3.1.2, deve esistere un cammino da s a qualsiasi nodo, inclusi quelli del ciclo. Ma poiché deve esistere un cammino da s a v_i , allora esiste un cammino da s a v_i che passa attraverso il ciclo, il che implica che esiste un cammino da s a v_i che contiene un ciclo. Questo è una contraddizione, poiché un cammino non può contenere un ciclo. Quindi, il sottografo T deve essere aciclico. □

Teorema 3.1.4. *Il protocollo SHOUT costruisce un albero ricoprente del grafo G .*

Dimostrazione. Dal Lemma 3.1.2 sappiamo che il sottografo T è connesso e include tutti i nodi di V , e dal Lemma 3.1.3 sappiamo che T è aciclico. Quindi, combinando questi due risultati, possiamo concludere che T è un albero ricoprente del grafo G . □

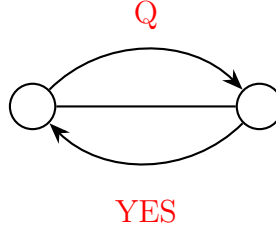
3.1.3 Message Complexity

Come menzionato sopra, il protocollo SHOUT = Flood + Reply. Quindi la complessità in termini di messaggi consiste nell'esecuzione di Flood(Q) più le risposte YES o NO per ogni richiesta ricevuta. In altre parole,

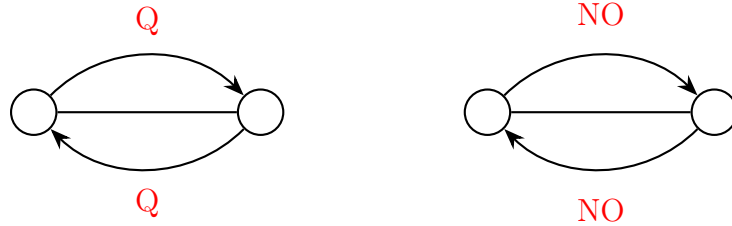
$$M[\text{SHOUT}] = M[\text{Flood}(Q) + \text{Reply}] = 2M[\text{Flood}(Q)] = O(4|E|) = O(|E|).$$

Più precisamente, i messaggi inviati dal protocollo possono essere divisi in due categorie: Q e Reply = { YES, NO }. Di conseguenza, abbiamo le seguenti casistiche:

- Per ogni arco $(x, y) \in ST$ costruito dal protocollo passano due messaggi: un messaggio Q inviato da x a y e un messaggio YES inviato da y a x . Il numero di archi sono $|E'| = |V| - 1$, quindi il numero di messaggi in questa categoria è $2(|V| - 1) = 2(n - 1)$.



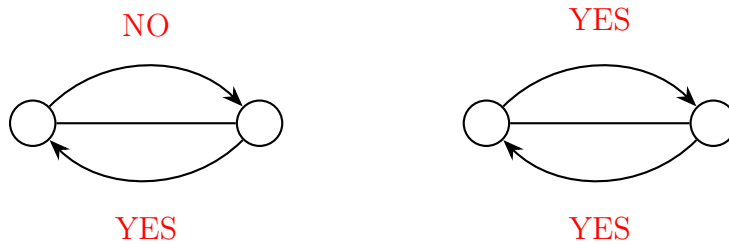
- Per ogni arco $(x, y) \in E \setminus ST$ passano quattro messaggi: due messaggi Q in entrambe le direzioni, seguiti da due messaggi NO in entrambe le direzioni. Il numero di archi in questa categoria è $|E| - |E'| = |E| - (|V| - 1)$, quindi il numero di messaggi in questa categoria è $4(|E| - (|V| - 1)) = 4(m - n + 1)$.



Osservazione. Su ogni arco (x, y) non potranno mai passare le coppie di messaggi (NO, YES) e (YES, YES).

(NO, YES) Non può accadere in quanto il nodo sinistro per rispondere a NO deve prima aver ricevuto una richiesta dal nodo destro, ma per inviare una richiesta, il nodo destro deve avere già un padre, dunque non può rispondere con YES a nessuna richiesta.

(YES, YES) Non può accadere in quanto due nodi per inviarsi reciprocamente un messaggio YES devono essersi entrambi inviati reciprocamente un messaggio Q, ma ciò implica che hanno già un padre, e quindi hanno già risposto con un messaggio YES a un altro nodo.



Quindi, il numero totale di messaggi è:

$$M[\text{SHOUT}] = 2(n - 1) + 4(m - n + 1) = 4m - 2n + 2 = O(|E|).$$

3.2 Single Source SHOUT+ Protocol

Si osserva che nel protocollo SHOUT, i messaggi di risposta NO non sono necessari per far sapere al nodo che ha inviato la richiesta quando entrare nello stato **done**. Infatti, se un nodo y invia NO ad x , allora y ha inviato un messaggio Q ad x prima dell'invio di NO, in quanto y per inviare NO deve trovarsi nello stato **active**, e per trovarsi nello stato **active**, il nodo y deve avere già scelto un nodo come padre e aver mandato il messaggio Q a tutti i suoi vicini. Dunque il nodo x può interpretare il messaggio Q inviato da y come un messaggio NO, e quindi non è necessario inviare un messaggio NO esplicito.

In questa maniera, la message complexity del protocollo SHOUT si riduce a

$$M[\text{SHOUT+}] = 2(n - 1) + 2(m - n + 1) = 2m = O(|E|).$$

Riguardo la terminazione locale del protocollo, ogni nodo sa che il suo compito è terminato nel momento in cui passa nello stato **done**. Sarebbe interessante sapere se un nodo è in grado di capire quanto il processo termina globalmente, cioè quando tutti i nodi sono nello stato **done**. Certamente, se la sorgente è s è in grado di sapere quanto lo spanning tree è completato, allora potrebbe mandare una notifica in broadcast a tutti i nodi e avvertirli della terminazione globale. Il problema è che ogni nodo ha solamente una **visione locale** della rete in cui si trova, perciò a meno che non ci sia un nodo che riesca ad osservare lo stato della rete dall'alto non è possibile sapere solo col protocollo SHOUT/SHOUT+ quando il processo è terminato globalmente.

Osservazione. Osserviamo che anche nel caso della costruzione dello spanning tree, il lower bound $\Omega(m)$ è rispettato, in quanto per costruire lo spanning tree è necessario che ogni nodo riceva almeno un messaggio da ogni vicino, e quindi è necessario che ogni arco venga attraversato da almeno un messaggio.

Osservazione. Si può osservare che lo spanning tree costruito dal protocollo SHOUT/SHOUT+ dipende fortemente dalla singola esecuzione e non dal protocollo, in quanto i ritardi di trasmissione variano ogni volta. Ciò implica che maneggiando adeguatamente i ritardi di trasmissione potrebbe accadere che $\text{diam}(ST) \gg \text{diam}(G)$.

Teorema 3.2.1. *L'esecuzione di ogni protocollo di Broadcast costruisce uno spanning tree entrante (ogni nodo possiede solamente informazioni su chi è il nodo padre, generando un albero orientato verso la radice) del grafo G .*

Dimostrazione. Diamo un'idea della dimostrazione. Definiamo

$$\text{father}(x) = \text{il nodo dal quale } x \text{ ha ricevuto per primo il messaggio } m$$

Allora, se consideriamo l'ordinamento parziale $\text{father}(x) > x$ otteniamo uno spanning tree entrante, con radice s . $\square$

Algorithm 3 Algoritmo di SHOUT

```
if state(x) = initiator then spontaneously
    root  $\leftarrow$  true
    tree_neigh(x)  $\leftarrow$   $\emptyset$ 
    state(x)  $\leftarrow$  active
    counter  $\leftarrow$  0
    for all  $v \in N(x)$  do
        send(Q, v)
    end for
end if

if state(x) = idle then
    Q  $\leftarrow$  receive()
    tree_neigh(x)  $\leftarrow$  {sender}
    root  $\leftarrow$  false
    counter  $\leftarrow$  1
    send(YES, sender)
    if counter =  $|N(x)|$  then
        state(x)  $\leftarrow$  done
    else
        for all  $v \in N(x) \setminus \{\text{sender}\}$  do
            send(Q, v)
        end for
        state(x)  $\leftarrow$  active
    end if
end if

if state(x) = active then
    reply  $\leftarrow$  receive()
    if reply = YES then
        tree_neigh(x)  $\leftarrow$  tree_neigh(x)  $\cup$  {sender}
        counter  $\leftarrow$  counter + 1
        if counter =  $|N(x)|$  then
            state(x)  $\leftarrow$  done
        end if
    end if
end if

    if reply = Q then
        counter  $\leftarrow$  counter + 1
        if counter =  $|N(x)|$  then
            state(x)  $\leftarrow$  done
        end if
    end if
end if
```

3.3 Multiple Sources Spanning Tree Construction

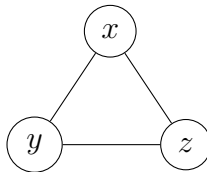
Supponiamo ora di avere più nodi *initiator*, ossia che

$$\exists I \subseteq V, |I| > 1 \text{ e } \forall s \in I, \text{state}(s) = \text{initiator}.$$

In questo scenario, l'insieme delle restrizioni diventa $R = \{\text{TR}, \text{BL}, \text{CN}\}$

Teorema 3.3.1. *Sotto le assunzioni $R = \{\text{TR}, \text{BL}, \text{CN}\}$, il problema della costruzione di spanning tree con multipli *initiator* non è deterministicamente risolvibile.*

Dimostrazione. Consideriamo una clique di 3 nodi x, y, z , tutti e tre *initiator*. Consideriamo il caso sincrono in cui tutti si attivano e iniziano il protocollo al tempo t_0 .



Sappiamo che se prendiamo dei nodi aventi lo stesso stato interno $\sigma(v, t)$, allora essi da quel momento in poi si comporteranno allo stesso modo, dato che nessuno è in grado di distinguere una loro *identità* all'interno della rete. In questo caso, al tempo t_0 , abbiamo che $\sigma(x, t_0) = \sigma(y, t_0) = \sigma(z, t_0) = \text{initiator}$, e quindi x, y, z si comporteranno allo stesso modo. Quindi ciascun nodo invierà un messaggio Q ed entrerà nello stato *active*. Quando riceveranno un messaggio Q da ciascuno dei loro vicini, risponderanno con un messaggio NO a ciascuno dei loro vicini, e quindi entreranno nello stato *done*. Quindi, alla fine dell'esecuzione, avremo che

$$\text{tree_neigh}(x) = \text{tree_neigh}(y) = \text{tree_neigh}(z) = \emptyset,$$

e quindi non è stato costruito alcun albero ricoprente del grafo G . □

Per risolvere questo problema è necessario risolvere un'altro problema: il problema della **leader election**. Eletto un leader tra tutti i nodi *initiator*, esso potrà eseguire uno dei protocolli di costruzione di spanning tree a singola sorgente già noti.

Vedremo in futuro che il problema della leader election può essere risolto in maniera deterministica solamente se i nodi sono univocamente identificati in maniera globale

Capitolo 4

Wake-Up Problem

Il problema del wake-up è un problema nel mondo dei sistemi distribuiti che riguarda un insieme di nodi iniziali *svegli (attivi)* che devono svegliare un insieme di nodi *dormienti (inattivi)* tramite lo scambio di messaggi. Il problema del wake-up è una generalizzazione del problema del broadcast, in cui non c'è un solo nodo **initiator**, bensì un sottoinsieme di nodi.

Formalmente, il problema del wake-up può essere definito come segue, dalla quadrupla $\langle P_{init}, P_{final}, R, \Sigma \rangle$, e sia $W \subseteq V$ l'insieme di **initiator** tale che $W \neq \emptyset$:

$$P_{init} = \forall x \in V$$

$$state(x) = \begin{cases} \text{awake} & \text{if } x \in W \\ \text{asleep} & \text{if } x \in V \setminus W \end{cases}$$

$$P_{final} = \forall x \in V, state(x) = \text{awake}$$

$$R = \{\text{TR}, \text{BL}, \text{CN}\}$$

$$\Sigma = \{\text{awake}, \text{asleep}\}$$

Osservazione. Se il numero di nodi $|V| = n$, allora il numero possibile di configurazioni iniziali $|P_{init}| = 2^n - 1$ (insieme vuoto). Quindi l'avversario è in grado di scegliere tra $2^n - 1$ configurazioni iniziali una volta visto il grafo della rete, oltre a poter ritardare a piacere le trasmissioni.

Algorithm 4 Algoritmo WFlood

```
if state(x) = awake then spontaneamente
  for all y ∈ N(x) do
    send(wake-up, y)
  end for
end if
if state(x) = asleep ∧ receive(wake-up, sender) then
  state(x) ← awake
  for all z ∈ N(x) \ {sender} do
    send(wake-up, z)
  end for
end if
```

4.1 Analisi dell'algoritmo WFlood

4.1.1 Correttezza

La correttezza dell'algoritmo WFlood segue dalla correttezza dell'algoritmo di flooding. Di seguito è riportata una dimostrazione formale:

Teorema 4.1.1. *Esiste un tempo $t \in \mathbb{R}^+$ tale che $\forall x \in V, state(x) = \text{awake}$, ossia che al tempo t , $W \equiv V$.*

Dimostrazione. Consideriamo $W \subset V$ sottoinsieme proprio. Se $W = V$, allora il tempo di terminazione dell'algoritmo è $t = 0$. Definiamo ora la distanza di un nodo dall'insieme W come

$$d(x, W) = \min_{y \in W} d(x, y)$$

Per definizione del protocollo, sappiamo che a un certo punto tutti i nodi che sono nello stato **awake** avranno inviato il messaggio a tutti i vicini, e data l'assunzione di TR, possiamo essere certi che esiste un tempo finito t_1 entro il quale tutti i nodi a distanza 1 da W avranno ricevuto il messaggio. Definiamo quindi l'insieme

$$L_1 = \{x \in V : d(x, W) = 1\}$$

Supponiamo per *ipotesi induttiva* che questo valga per una qualsiasi distanza $d > 1$, ossia che esiste un tempo t_d tale che tutti i nodi in L_d sono nello stato **awake**. Consideriamo un nodo $x \in L_{d+1}$. Osserviamo che poichè $x \in L_{d+1}$, allora deve esistere un nodo $y \in L_d$ tale per cui $(y, x) \in E$. Se così non fosse, allora ogni cammino minimo da W arriverebbe in x tramite un arco (z, x) tale che $z \in L_k$ con $k \neq d$, e ciò implica che x è a distanza $k + 1 \neq d + 1$, ovvero che $x \notin L_{d+1}$.

Dato che y è stato informato entro il tempo t_d , per definizione del protocollo y invierà **wake-up** a tutti i suoi vicini, compreso x . Grazie alla TR, siamo certi che questo messaggio arriverà entro un tempo finito t_{d+1} . Quindi possiamo concludere che $\exists t_{d+1}$ tempo finito entro il quale $\forall x \in L_{d+1}, state(x) = \text{awake}$. $\square$

4.1.2 Message Complexity

Sia $k = |W|$ il numero di nodi iniziali nello stato **awake**. Ogni nodo in W invia un messaggio a tutti i suoi vicini, mentre ogni nodo in $V \setminus W$ invia un messaggio a tutti i suoi vicini eccetto il **sender**. Dunque, il numero totali di messaggi sono:

$$\begin{aligned} M[\text{WFlood}] &= \sum_{u \in W} |N(u)| + \sum_{u \in V \setminus W} (|N(u)| - 1) \\ &= \sum_{u \in W} |N(u)| + \sum_{u \in V \setminus W} |N(u)| - \sum_{u \in V \setminus W} 1 \\ &= \sum_{u \in V} |N(u)| - \sum_{u \in V \setminus W} 1 \\ &= 2m - (n - k) = 2m + n - k \end{aligned}$$

Osservazione. Osserviamo che il caso peggiore è quando $k = n$, ossia che $W \equiv V$. In questo caso il numero di messaggi è esattamente $2m$. Viceversa, il caso migliore è quando il $|W| = 1$ (come nel broadcast) e il numero di messaggi è $2m - n + 1$. Vale quindi la seguente relazione:

$$M[\text{WFlood}/R] \geq M[\text{Broadcast}R_{\text{bcst}}] = \Omega(m)$$

Questo dimostra ancora che il problema del **wake-up** è una generalizzazione del problema del broadcast.

4.1.3 Ideal Time Complexity

Consideriamo il nuovo insieme di restrizioni

$$R = R \cup \{\text{Synchronized Clock, Bounded Communication Delay}\}$$

allora possiamo calcolare il tempo di terminazione ottimale. Abbiamo osservato che il caso $|W| = n$ rappresenta il worst-case scenario nel caso della message complexity. Per quanto riguarda il costo temporale, il caso peggiore è quando $|W| = 1$, infatti in questo caso abbiamo che

$$T[\text{Broadcast}R_{\text{bcst}}] = T[\text{WFlood}/R] = O(\text{diam}(G))$$

Osservazione. Se consideriamo il problema del wake-up su un albero, allora è vero che

$$M[\text{WFlood}/\text{Tree}] = 2m - n + k = 2(n - 1) - n + k = n + k - 2$$

4.2 Lower Bound su Clique (Da FARE)

Capitolo 5

Labeled Hypercube

Generalmente, in un contesto reale si vuole costruire reti che abbiano le seguenti proprietà:

- **Diametro piccolo**, $O(1)$ oppure $\text{PolyLog}(n)$, in modo da ridurre i tempi di comunicazione tra i nodi.
- **Grado piccolo**, in modo da ottenere grafi **sparsi**, che sono più facili da costruire e mantenere.

Il principale vantaggio di un grafo sparso è la sua **scalabilità**, ovvero la capacità di crescere senza un aumento troppo grande del numero di link. Per esempio una *clique* non è molto scalabile, perché se aggiunto un nuovo nodo, è necessario aggiungere altri $n - 1$ link. Al contrario, se consideriamo un grafo a *lista*, osserviamo che è facilmente scalabile, in quanto il grado di ciascun nodo è 2, indipendentemente dal numero di nodi presenti. Tuttavia, il diametro di un grafo a lista è $O(n)$ (mentre il diametro di una clique è $O(1)$), il che lo rende poco efficiente per la comunicazione tra nodi distanti.

Quindi osserviamo che esiste un trade-off tra diametro e grado: se vogliamo un diametro piccolo, dobbiamo accettare un grado più grande, e viceversa. Un grafo ideale sarebbe quello che riesce a mantenere sia un diametro piccolo che un grado piccolo.

Consideriamo un altro esempio di grafo, il *grafo a griglia*. In questo grafo, i nodi sono disposti in una **griglia bidimensionale**, e ogni nodo è connesso ai suoi vicini immediati. Il grado di ogni nodo è al massimo 4 (a meno che non si trovi ai bordi della griglia), e il diametro del grafo è $O(\sqrt{n})$, il che è un miglioramento rispetto al grafo a lista, ma ancora non è ideale.

5.1 Broadcast su Labeled Hypercube

Vogliamo quindi studiare il problema del broadcast in un grafo che abbia sia un diametro piccolo che un grado piccolo. Un esempio di tale grafo è l'**ipercubo etichettato** (labeled hypercube), che ha un diametro di $O(\log n)$ e un grado di $O(\log n)$, rendendolo una scelta interessante per la costruzione di reti scalabili ed efficienti.

Consideriamo l'insieme delle restrizioni

$$R = \{\text{TR}, \text{BL}, \text{CN}, \text{UI}, \text{KN}\}$$

dove KN è l'assunzione di **knowledge**, ovvero che ogni nodo la struttura della rete in cui si trova, ossia un **labeled hypercube** di dimensione d .

5.1.1 Costruzione Hypercube

Definizione 5.1.1 (Hamming Distance). La **Hamming distance** tra due stringhe binarie $\bar{x}$ e $\bar{y}$ di lunghezza d , denotata come $h_d(\bar{x}, \bar{y})$, è il numero di posizioni in cui le due stringhe differiscono. Formalmente, se $\bar{x} = x_1x_2 \dots x_d$ e $\bar{y} = y_1y_2 \dots y_d$, allora:

$$h_d(\bar{x}, \bar{y}) = |\{i : 1 \leq i \leq d, x_i \neq y_i\}|$$

Definizione 5.1.2 (Labeled Hypercube). Definiamo la struttura **labeled hypercube** $\mathcal{H}_d$ di dimensione d come un grafo $\mathcal{H}_d(V, E)$, dove:

- $V = \{0, 1\}^d \implies n = |V| = 2^d$. Ciascun nodo è univocamente identificato da una stringa (**label**) binaria di lunghezza d .
- $E = \{(\bar{x}, \bar{y}) : \bar{x}, \bar{y} \in V : h_d(\bar{x}, \bar{y}) = 1\} \implies m = |E| = \frac{n \cdot d}{2} = \frac{n \log_2 n}{2}$. Due nodi sono connessi da un arco se e solo se differiscono in esattamente una posizione (ovvero, hanno un Hamming distance di 1).

Inoltre, ogni arco è etichettato con l'indice della posizione in cui i due nodi differiscono. Ad esempio, se $\bar{x} = 000$ e $\bar{y} = 001$, allora l'arco tra $\bar{x}$ e $\bar{y}$ è etichettato con 3, poiché differiscono solo nella terza posizione.

Diremo che $\mathcal{H}_d$ è una struttura quasi sparsa, poiché il numero di archi è $O(n \log n)$, che è maggiore di $O(n)$ (struttura sparsa) ma ancora molto più piccolo di $O(n^2)$ (struttura densa).

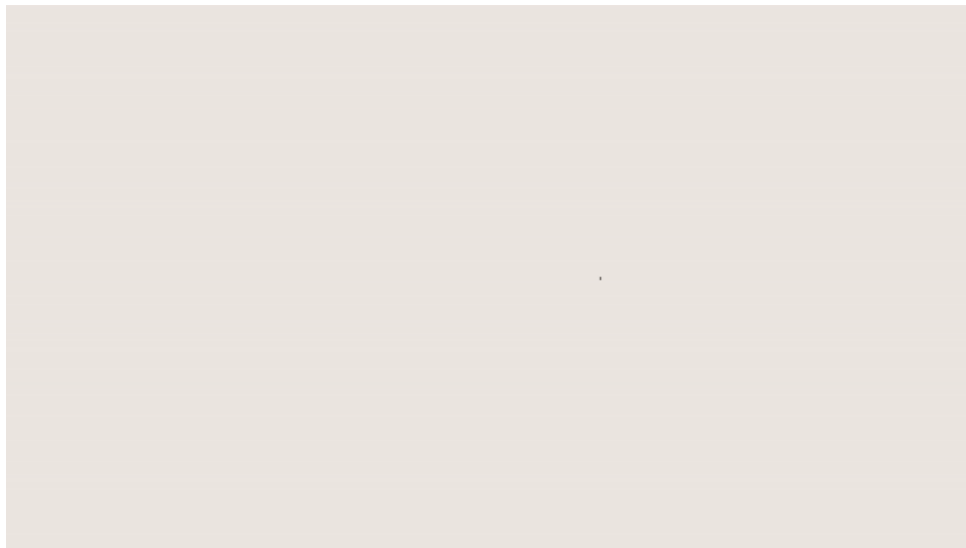
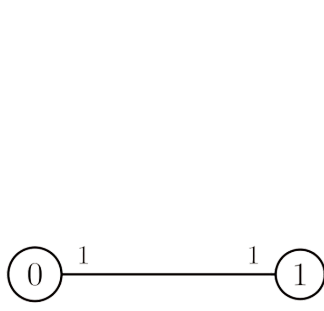
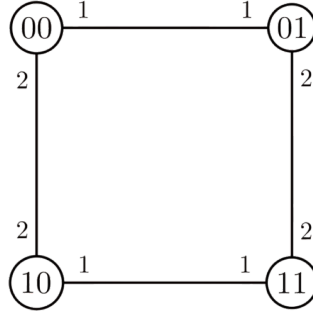


Figura 5.1: Animazione di un labeled hypercube di dimensione $d = 3$. Aprire con Adobe Reader oppure Okular.

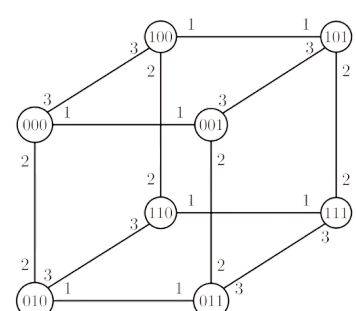
Osservazione. Il labeled hypercube è un grafo molto scalabile. Infatti si può estendere un ipercubo di dimensione d con un altro ipercubo di dimensione d , semplicemente aggiungendo un arco tra i nodi analoghi dei due ipercubi.



(a) Labeled Hypercube di dimensione $d = 1$.



(b) Labeled Hypercube di dimensione $d = 2$.



(c) Labeled Hypercube di dimensione $d = 3$.

Teorema 5.1.1. *Sia $\mathcal{H}_d$ un labeled hypercube di dimensione d . Allora:*

1. $\delta(\bar{x}) = d = \log_2 n \quad \forall \bar{x} \in V$
2. $\text{diam}(\mathcal{H}_d) = d = \log_2 n$

Dimostrazione. Dimostriamo ciascuna affermazione separatamente:

1. Poichè $|V| = 2^d$, e ogni nodo ha come vicini a distanza di hamming 1, allora ogni nodo ha esattamente d vicini.
2. Per dimostrare che il diametro è d , dobbiamo mostrare che $\forall \bar{x}, \bar{y} \in V$, esiste un cammino $P = (\bar{x} = \bar{v}_0, \bar{v}_1, \dots, \bar{v}_k = \bar{y})$ tale che $k \leq d$. Siano quindi $\bar{x} = x_1 x_2 \dots x_d$ e $\bar{y} = y_1 y_2 \dots y_d$ due nodi arbitrari in V .
 - Se $\bar{x} = \bar{y}$, allora $k = 0 \leq d$.
 - Se $\bar{x} \neq \bar{y}$, scorrendo gli indici delle due stringhe a partire dalla posizione più significativa (MSB, da sinistra), e confrontando i bit, incontreremo il primo indice j_1 tale per cui $x_{j_1} \neq y_{j_1}$. A questo punto, per costruzione dell'ipercubo, esisterà un suo vicino $\bar{x}'$ tale che $x'_{j_1} = y_{j_1}$ e $x'_i = x_i$ per tutti gli altri indici i .

$$\bar{x} = x_1 x_2 \dots x_{j_1} \dots x_d \xrightarrow{(j_1)} \bar{x}' = x_1 x_2 \dots y_{j_1} \dots x_d$$

Consideriamo quindi l'arco $(\bar{x}, \bar{x}')$ come primo arco del cammino P .

A questo punto, $\forall i \leq j_1, x'_i = x_i$, quindi partendo dall'indice $j_1 + 1$ e continuando a scorrere verso sinistra, se arrivati a d e non abbiamo ancora trovato un indice j_2 tale che $x'_{j_2} \neq y_{j_2}$, allora $\bar{x}' = \bar{y}$ e il cammino P è completo con $k = 1 \leq d$.

Altrimenti, se esiste un indice j_2 tale che $x'_{j_2} \neq y_{j_2}$, allora esisterà un vicino $\bar{x}''$ di $\bar{x}'$ tale che $x''_{j_2} = y_{j_2}$ e $x''_i = x'_i$ per tutti gli altri indici i .

Iterando questo processo, alla fine si arriverà ad un nodo $\bar{x}^{(k)}$ tale che $\bar{x}^{(k)} = \bar{y}$, e il cammino P sarà completo con $k \leq d$, poiché in ogni passo ci spostiamo di un bit solo, e al massimo ci sono d bit da modificare per trasformare $\bar{x}$ in $\bar{y}$.

□

Osservazione. Se ci spostiamo da MSB $\rightarrow$ LSB, allora il cammino P che abbiamo costruito è un cammino **crescente**, ovvero un cammino in cui le etichette degli archi sono

in ordine crescente. Se invece ci spostiamo da $\text{LSB} \rightarrow \text{MSB}$, allora il cammino P è un cammino **decescente**, ovvero un cammino in cui le etichette degli archi sono in ordine decrescente.

5.2 Protocollo Hyperflood

Il protocollo **Hypercube Flooding** (Hyperflood) è un algoritmo di broadcast che sfrutta la struttura del labeled hypercube per diffondere un messaggio da un nodo sorgente a tutti gli altri nodi in modo efficiente. Inoltre, viene altamente sfruttata la conoscenza della struttura del grafo (assunzione KN) per ottimizzare la diffusione del messaggio.

L'idea del protocollo è la seguente: l'unico nodo $\bar{s} \in V$ **initiator** invia il messaggio m a tutti i suoi vicini. Qualsiasi altro nodo $\bar{x} \neq \bar{s}$ riceve il messaggio da un arco etichettato con l , allora inoltrerà il messaggio su tutti gli archi

$$\{(\bar{x}, \bar{y}) \in E : l' = \lambda(\bar{x}, \bar{y}) < l\}$$

Osservazione. Il protocollo sfrutta la struttura gerarchica del labeled hypercube: il nodo iniziatore invia il messaggio ad un nodo all'interno di ogni sotto-iperubo di dimensione $l' = l - 1$, e ognuno di questi nodi diventa iniziatore del sotto-iperubo di cui fa parte.

Algorithm 5 Hyperflood

```

if state( $\bar{x}$ ) = state( $\bar{s}$ ) then spontaneamente
  for all  $\bar{y} \in N(\bar{x})$  do
    send( $m, \bar{y}$ )
  end for
else
   $m, l \leftarrow \text{receive}()$ 
  for all  $\bar{y} \in N(\bar{x})$  do
    if  $\lambda(\bar{x}, \bar{y}) < l$  then
      send( $m, \bar{y}$ )
    end if
  end for
end if

```

Teorema 5.2.1 (Correttezza). *Per ogni coppia di nodi $\bar{x}, \bar{y} \in V$, esiste sempre un cammino P con **etichette decrescenti** da $\bar{x}$ a $\bar{y}$. Nel protocollo Hyperflood, il messaggio verrà inoltrato lungo questo cammino.*

Dimostrazione. La dimostrazione è la stessa del secondo punto del teorema precedente, in quanto il protocollo Hyperflood costruisce esattamente un cammino con etichette decrescenti da $\bar{x}$ a $\bar{y}$.

Vediamo invece un esempio pratico. Consideriamo due nodi $\bar{x} = 100010100$ e $\bar{y} = 110001000$. Consideriamo la procedura per costruire un cammino usata nel secondo punto del teorema precedente, ma partendo da $\text{LSB} \rightarrow \text{MSB}$, in modo da ottenere un cammino con etichette decrescenti. Consideriamo inoltre gli indici ordinali in modo decrescente, allora i due nodi differiscono in posizioni 8, 5, 4, 3. Allora, un cammino che li collega è formato dagli archi con etichette 8, 5, 4, 3, ovvero con etichette decrescenti.

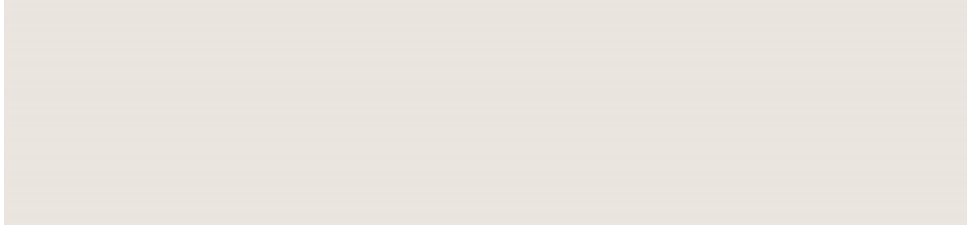


Figura 5.3: Animazione costruzione cammino. Aprire con Adobe Reader oppure Okular.

□

Teorema 5.2.2 (Message Complexity).

$$M[\textit{Hyperflood}] = n - 1 = O(n)$$

Dimostrazione. Per dimostrare che il numero di messaggi è $n - 1$, ossia uno per ogni nodo, mostreremo che il grafo indotto dai archi lungo cui viene trasmesso il messaggio è un **albero**.

Supponiamo per assurdo che esiste un nodo $\bar{y}$ che riceve due copie del messaggio. Allora nel grafo delle comunicazioni esistono due cammini *disgiunti* che partono da un nodo $\bar{x}$ (sorgente inclusa) e arrivano a $\bar{y}$.

Per costruzione dell'ipercubo, i due cammini sono tali per cui i loro rispettivi archi iniziali posseggono due etichette $l_1 \neq l_2$. Senza perdita di generalità, assumiamo $l_1 < l_2$ e che l'arco a sinistra inizi con l'arco etichettato l_1 e quello a destra inizi con l'arco etichettato l_2 . Sia $\bar{x}'$ il nodo raggiunto da l_1 e $\bar{x}''$ il nodo raggiunto da l_2 , come in [Figura 5.4](#).

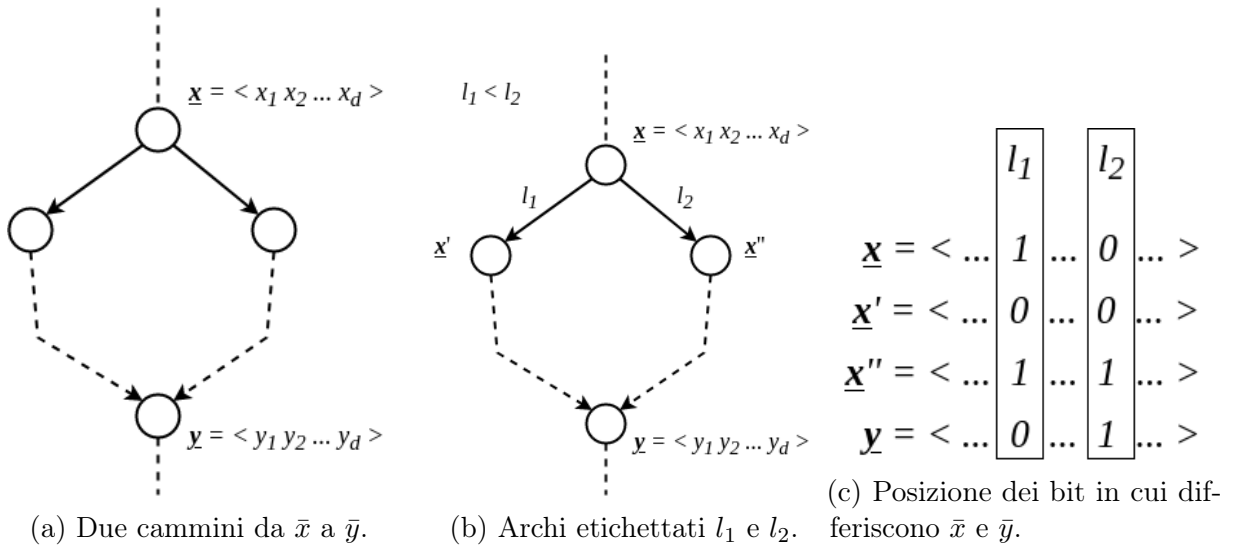


Figura 5.4: Dimostrazione della struttura ad albero nel protocollo Hyperflood.

Per costruzione del cammino, il nodo $\bar{x}$ differisce da $\bar{y}$ almeno per i bit in posizione l_1 e l_2 (vedere animazione in [Figura 5.3](#)).

Abbiamo quindi che $\bar{x}'$ differisce da $\bar{y}$ per il bit in posizione l_2 , e $\bar{x}''$ differisce da $\bar{y}$ per il bit in posizione l_1 .

Ci troviamo quindi nella situazione in [Figura 5.4c](#). Poiché i cammini procedono per etichette **decrescenti**, il cammino di sinistra includerà solo i nodi che differiscono da $\bar{x}'$ per i bit in posizione $l_i < l_1$, ma questo implica che il cammino di sinistra porta a $\bar{y}$ senza mai cambiare il bit in posizione l_2 , ma questo è un **assurdo** dato che $\bar{x}$ e $\bar{y}$ differiscono per il bit in posizione l_2 .

Questo dimostra che nel grafo di comunicazione non ci sono cicli, e quindi è un albero. Di conseguenza, ogni nodo riceve esattamente un messaggio, e il numero totale di messaggi è $n - 1$. $\square$

Teorema 5.2.3 (Ideal Time). *Sotto le restrizioni*

$$R \cup \{\textit{Synchronized Clock}, \textit{Bounded Communication Delay}\}$$

il tempo ideale per il broadcast è $O(\log n)$.

Dimostrazione. Dimostrazione per induzione sulla *distanza di Hamming* dei nodi dalla sorgente. Dimostriamo che per ogni $j \geq d$, e per ogni $\bar{x}$ a distanza di Hamming j dalla sorgente, il nodo $\bar{x}$ riceve il messaggio al tempo j .

$$\forall j \in [d], \forall \bar{x} \in V = \{0, 1\}^d : h_d(\bar{s}, \bar{x}) = j \implies t_{\text{recv}}(\bar{x}) = j$$

Per $j = 1$ è banale, in quanto per definizione del protocollo, la sorgente invia il messaggio a tutti i suoi vicini i quali hanno distanza di Hamming 1 dalla sorgente, e quindi ricevono il messaggio al tempo $t = 1$.

Definiamo ora $L_j = \{\bar{x} \in V : h_d(\bar{s}, \bar{x}) = j\}$ e supponiamo per induzione che sia vero per $j - 1$, ovvero che tutti i nodi in L_{j-1} ricevano il messaggio al tempo $t = j - 1$. Sappiamo che per ogni nodo $\bar{x} \in L_j$, esiste un cammino con etichette decrescenti da $\bar{s}$ a $\bar{x}$, allora esiste un nodo $\bar{y} \in L_{j-1}$ tale che $\bar{x}$ è un vicino di $\bar{y}$. Per ipotesi, $\bar{y}$ è già stato informato al tempo $t = j - 1$ e inoltre il messaggio impiega un istante per attraversare l'arco $(\bar{y}, \bar{x})$, quindi $\bar{x}$ riceve il messaggio al tempo $t = j$. Poiché la lunghezza massima di un cammino è d , allora ogni nodo riceve il messaggio al tempo $t \leq d = O(\log n)$. $\square$

Capitolo 6

Depth First Traversal (DFT) Protocol

Consideriamo un problema in cui uno *iniziale* possiede un **token**, e di volere che questo token (non replicabile) venga passato a **tutti** i nodi della rete. Possiamo vedere questo problema come un problema di *scheduling/assegnazione* di un compito/risorsa (il token) a più processori (i nodi della rete), dove la risorsa è *mutualmente esclusiva* tra i nodi.

Formalmente, il problema è definito dalla seguente tripla $\langle P_{init}, P_{final}, R \rangle$, dove, sia per ogni nodo $x \in V$, **received**(x) un registro interno che contiene **true** se x ha ricevuto il token almeno una volta, e **false** altrimenti:

- P_{init} : $\exists! x \in V$ tale che **received**(x) = **true** e per ogni $y \in V$ con $y \neq x$, **received**(y) = **false**, ossia x è il nodo iniziale che possiede il token.
- P_{final} : per ogni $x \in V$, **received**(x) = **true**, ossia tutti i nodi hanno ricevuto il token almeno una volta seguendo l'ordine di una DFS.
- $R = \{\text{TR}, \text{BL}, \text{CN}, \text{UI}\}$.

6.1 Protocollo BACK

Il protocollo più semplice per risolvere questo problema è il protocollo BACK, che funziona come segue:

1. Quando un nodo riceve il token per la *prima volta*, ricorda il nodo che gli ha inviato il token (il padre) e inoltra il token ad uno dei suoi vicini non visitati con il messaggio **token**, aspettando la sua risposta.
 2. Quando il vicino riceve il token, se lo ha già ricevuto lo ritorna immediatamente con il messaggio **back_edge**, altrimenti lo inoltra a tutti i suoi vicini in modo sequenziale, per poi ritornarlo al nodo da cui lo ha ricevuto con il messaggio **return**.
 3. Alla ricezione di un **return**, il nodo inoltra il token al vicino successivo non visitato, se esiste, altrimenti lo ritorna al padre con il messaggio **return**.
- $S = \{\text{initiator}, \text{idle}, \text{visited}, \text{done}\}$
 - $S_{init} = \{\text{initiator}, \text{idle}\}$
 - $S_{final} = \{\text{done}\}$

Algorithm 6 Protocollo BACK

```
1: if state(x) = initiator then spontaneously
2:   unvisited  $\leftarrow N(x)$ 
3:   initiator  $\leftarrow True$ 
4:   visit()
5: end if
6:
7: if state(x) = idle then
8:   token  $\leftarrow receive()$ 
9:   parent  $\leftarrow sender(token)$ 
10:  unvisited  $\leftarrow N(x) \setminus \{parent\}$ 
11:  initiator  $\leftarrow False$ 
12:  visit()
13: end if
14:
15: if state(x) = visited then
16:   msg  $\leftarrow receive()$ 
17:   if msg = token then
18:     unvisited  $\leftarrow N(x) \setminus \{sender(msg)\}$ 
19:     send(back_edge, sender(msg))
20:   else if msg = return then
21:     visit()
22:   else if msg = back_edge then
23:     visit()
24:   end if
25: end if
26:
27: if state(x) = done then
28:   do_nothing()
29: end if
30:
31: procedure VISIT
32:   if unvisited  $\neq \emptyset$  then
33:     next  $\leftarrow pop(unvisited)$ 
34:     send(token, next)
35:     state(x)  $\leftarrow visited$ 
36:   else
37:     if not initiator then
38:       send(return, parent)
39:     end if
40:     state(x)  $\leftarrow done$ 
41:   end if
42: end procedure
```

Teorema 6.1.1 (Message Complexity).

$$M[BACK] = 2m \in \theta(m)$$

Dimostrazione. Osserviamo che esistono 3 tipologie di messaggi:

- **token**, quando un nodo inoltra il token per procedere con la visita.
- **return**, quando un nodo già visitato restituisce il token al padre dopo aver visitato tutti i suoi vicini.
- **back_edge**, quando un nodo già visitato riceve il token e lo restituisce immediatamente al mittente.

Inoltre, su ogni arco può passare uno tra i messaggi **back_edge** e **return**, mentre il messaggio **token** può passare al massimo una volta per ogni arco, quando viene inoltrato per la prima volta. Di conseguenza, il numero totale di messaggi è al massimo $2m \in \theta(m)$. In realtà si dimostra che il lower bound rimane $\Omega(m)$, in quanto non è possibile effettuare il broadcasting di una informazione in meno di m messaggi. $\square$

Teorema 6.1.2 (Ideal Time). *Sotto le restrizioni*

$$R \cup \{Synchronized\ Clock, Bounded\ Communication\ Delay\}$$

si ha che

$$T[BACK] = \theta(m)$$

Dimostrazione. Ogni nodo esplora **sequenzialmente** i suoi vicini, e ogni messaggio impiega un tempo costante per essere consegnato. Di conseguenza, il tempo totale è proporzionale al numero di messaggi, che è $\theta(m)$. $\square$

6.2 Protocollo BACK+

Il principale difetto del protocollo BACK è che ogni nodo esplora i suoi vicini in modo sequenziale, e questo porta ad un tempo totale di $\theta(m)$. Il protocollo BACK+ migliora questo aspetto, permettendo ad ogni nodo di esplorare i suoi vicini in modo **parallelo**, e questo porta ad un tempo totale di $\theta(n)$, che è ottimale, aumentando la message complexity di un fattore 2.

- Quando un nodo x riceve il token, **informa** il suo vicinato $N(x)$ che possiede il token, ossia che è stato visitato.
- Tutti i nodi in $N(x)$ inviano un messaggio di **ack** a x per confermare che hanno ricevuto l'informazione.
- Dopo aver ricevuto tutti gli **ack**, x inoltra il token a uno dei suoi vicini non visitati.

La differenza sostanziale di questo protocollo è che i **back_edges** sono scoperti in parallelo e non più in modo sequenziale.

Teorema 6.2.1 (Message Complexity).

$$M[BACK+] = 4m \in \theta(m)$$

Dimostrazione. Osserviamo che esistono 4 tipologie di messaggi:

- **token**, quando un nodo inoltra il token per procedere con la visita.
- **visited**, quando un nodo che ha appena ricevuto il token lo notifica ai suoi vicini.
- **ack**, quando un nodo conferma di aver ricevuto l'informazione che il suo vicino è stato visitato.
- **return**, quando un nodo già visitato restituisce il token al padre dopo aver visitato tutti i suoi vicini.

Ogni nodo (eccetto la sorgente s) riceverà **token** una volta, e invierà **return** una sola volta, per un totale di $2(n-1)$ messaggi.

Per quanto riguarda il messaggio **visited**, ogni nodo lo invierà a tutto il suo vicinato meno che al nodo da cui hanno ricevuto il token. La sorgente sarà l'unico nodo che lo invierà a tutto il suo vicinato, quindi:

$$\begin{aligned}
M[\text{visited}] &= |N(s)| + \sum_{x \in V \setminus \{s\}} |N(x)| - 1 = \\
&= |N(s)| + \sum_{x \in V \setminus \{s\}} |N(x)| - \sum_{x \in V \setminus \{s\}} 1 = \\
&= \sum_{x \in V} |N(x)| - \sum_{x \in V \setminus \{s\}} 1 = 2m - (n-1)
\end{aligned}$$

Inoltre, per ogni notifica **visited** corrisponde un messaggio **ack**, quindi

$$M[\text{BACK+}] = 2(n-1) + 2(2m - (n-1)) = 4m = 2M[\text{BACK}] \in \theta(m)$$

□

Teorema 6.2.2 (Ideal Time). *Sotto le restrizioni*

$$R \cup \{\textit{Synchronized Clock, Bounded Communication Delay}\}$$

si ha che

$$T[\textit{BACK+}] = \theta(n)$$

Dimostrazione. Abbiamo che in soli due istanti, ogni nodo invierà le notifiche e riceverà le risposte. Per il tempo necessario al completamento del task sarà:

- $n-1$ per messaggi inviati di tipo **return**.
- $n-1$ per messaggi inviati di tipo **token**.
- n per messaggi inviati di tipo **visited**.
- n per messaggi inviati di tipo **ack**.

per un totale di $4n-2 \in \theta(n)$.

□

Osservazione. Con questa ottimizzazione, anche se si paga un fattore 2 alla message complexity, si ottiene un miglioramento significativo del tempo di esecuzione, che passa da $\theta(m)$ a $\theta(n)$, che è ottimale. In particolare, in grafi densi, dove m è dell'ordine di n^2 , questo miglioramento è particolarmente rilevante.

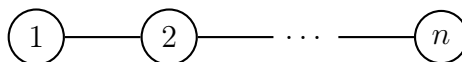
Capitolo 7

Tree Computations

Consideriamo adesso una serie di problemi computazionali su alberi. Come possiamo osservare, gli alberi hanno una struttura asimmetrica, e questo ci permette di risolvere alcuni problemi in modo più efficiente e semplice rispetto ai grafi generici o a struttura nelle quali è difficile rompere la simmetria. Infatti, non è necessario utilizzare tecniche di rottura della simmetria, come ad esempio l'assegnazione di ID unici ai nodi, per risolvere problemi su alberi.

Infatti, un nodo all'interno di albero può identificare la propria posizione all'interno della struttura,

- Un nodo sa di essere una *foglia* in quanto ha grado 1 ($N(x) = 1$), mentre è un nodo *interno* se ha grado maggiore di 1 ($N(x) > 1$).
- Un'altra proprietà importante è che ogni tipologia di albero ha almeno due foglie.



Introduciamo la tecnica di *saturazione*, che ci permette di risolvere problemi su alberi in ambiente distribuito in modo efficiente.

7.1 Saturation Technique

Consideriamo un ambiente distribuito con una topologia ad albero e sia tale informazione, riguardante la struttura topologica, nota a tutti i nodi della rete. La tecnica di saturazione consiste nell'identificare una singola coppia di nodi adiacenti all'interno della rete. Tale coppia di nodi sarà in grado di far iniziare altre computazioni. In pratica, la tecnica di saturazione consiste nell'eleggere un arco della rete. Osserviamo che non c'è nessuna garanzia su quale arco venga eletto: questa tecnica garantisce l'elezione di uno fra i possibili archi, ma non garantisce quale arco venga eletto.

Tipicamente la tecnica di saturazione è usata per risolvere problemi distribuiti di varia natura. In particolare, la saturazione è inserita all'interno di un processo più grande, che può essere diviso nelle seguenti tre fasi:

- **Attivazione:** in questa fase, i nodi iniziatori attivano i restanti nodi della rete utilizzando un protocollo di *wake-up*.

- **Saturazione:** in questa fase, a partire dalle foglie si identifica un'unica coppia di nodi adiacenti, anche detti *nodì saturati*.
- **Risoluzione:** in questa fase, i nodi saturati danno inizio ad una o più computazioni necessarie alla risoluzione di uno specifico problema distribuito.

Ci focalizziamo ora a definire le prime due fasi poiché la terza fase dipende dal problema specifico che si vuole risolvere. Definiamo la tripla $\langle P_{init}, P_{final}, R \rangle$ e sia $T = (V, E)$ l'albero considerato. Sia inoltre $W \subseteq V$, $W \neq \emptyset$ l'insieme dei nodi **initiator**. Allora

- P_{init} : $\forall x \in V$:,

$$status(x) = \begin{cases} \text{awake} & \text{se } x \in W \\ \text{asleep} & \text{altrimenti} \end{cases}$$
- P_{final} : $\exists!(u, v) \in E : status(u) = status(v) = \text{saturated}$ e $\forall w \in V \setminus \{u, v\} : status(w) = \text{active}$, ossia tutti i nodi tranne u e v sono attivi, mentre u e v sono saturati.
- $R = \{\text{TR}, \text{BL}, \text{CN}, \text{KT}, \text{MO}\}$, dove
KT = Knowledge of the Topology e MO = Message Ordering.

Siano

- $S = \{\text{available}, \text{active}, \text{processing}, \text{saturated}\}$ l'insieme degli stati dei nodi,
- $S_{init} = \{\text{available}\}$ l'insieme degli stati iniziali dei nodi,
- $S_{final} = \{\text{active}, \text{saturated}\}$ l'insieme degli stati finali dei nodi.

7.2 Minimum Finding

7.3 Eccentricities & Center Finding

Capitolo 8

Leader Election

In un sistema distribuito, è spesso necessario identificare un nodo che agisce da coordinatore di un determinato task. La presenza di tale entità può essere necessaria per *semplificare* lo svolgimento di un task oppure perché è richiesto per la natura stessa del problema.

Il problema di scegliere un leader è noto come *leader election* e consiste di scegliere un nodo che agisce da leader da un insieme omogeneo e simmetrico di individui. Spesso si fa riferimento a questo problema per **rompere la simmetria** del sistema.

Generalmente, il problema richiede di portare la rete da una configurazione iniziale in cui tutti i nodi sono in uno stesso stato, diciamo **asleep**, ad una configurazione finale in cui un nodo è in uno stato **leader** e tutti gli altri sono in uno stato **follower**.

Si può pensare alla leader election come un metodo che **forza** la restrizione **unique initiator**, infatti eletto il leader è possibile risolvere il task con protocolli a singolo iniziatore semplicemente ponendo il leader come nodo initiator.

Formalmente, possiamo definire il problema mediante la tripla $\langle P_{init}, P_{final}, R \rangle$

- $P_{init} : \forall x \in V : state(x) = \text{asleep}$
- $P_{final} : \exists! x \in V : state(x) = \text{leader} \wedge \forall y \in V \setminus \{x\} : state(y) = \text{follower}$
- $R = \{\text{TR}, \text{BL}, \text{CN}\}$

Teorema 8.0.1 (Impossibilità). *Sotto le restrizioni di R , è impossibile risolvere il problema di leader election in modo deterministico.*

Dimostrazione. La dimostrazione è identica a quella del teorema [Teorema 3.3.1](#).

Per avere un'idea, consideriamo un sistema con sole due entità x e y , inizialmente nello stesso stato **asleep**. Supponiamo l'esistenza di un protocollo Π , esso deve funzionare sotto qualsiasi condizione e ritardo dei messaggi. Consideriamo per semplicità un sistema sincrono, in cui x e y iniziano la computazione nello stesso istante. Poichè x e y sono indistinguibili, essi eseguiranno le stesse operazioni, invieranno gli stessi messaggi e riceveranno gli stessi messaggi. Di conseguenza, dopo un certo numero di round, x e y saranno ancora nello stesso stato, di conseguenza, se x diventa leader, allora anche y diventa leader, violando la condizione di unicità del leader. $\square$

È necessario ampliare l'insieme delle restrizioni R per poter risolvere il problema di leader election. Una restrizione banale è quella di **unique initiator**, che permette di risolvere il problema in modo banale.

Tuttavia, è possibile risolvere il problema assegnando un identificatore univoco a ciascun nodo e considerare come topologia di rete quella più simmetrica possibile, ovvero un anello.

Sia $G = (V, E)$ un anello con n (non noto) nodi, e sia $id : V \rightarrow \{1, 2, \dots, n\}$ una funzione che assegna un identificatore univoco a ciascun nodo. Ciascun nodo è a conoscenza del proprio identificatore e non conosce gli identificatori degli altri nodi.

Definizione 8.0.1 (Anello). Un anello è un grafo $G = (V, E)$ in cui $V = \{x_1, x_2, \dots, x_n\}$ e

$$E = \{(x_i, x_{i+1}) \in V^2 : 1 \leq i < n\} \cup \{(x_n, x_1)\}.$$

Osserviamo che in un anello, $|E| = |V| = n$. Inoltre, $\forall x \in V, \delta(x) = 2$.

Osservazione. Il numero n non è noto ai nodi, per semplicità, possiamo assumere che id sia una permutazione di $\{1, 2, \dots, n\}$, ma non è necessario, basta che sia una funzione iniettiva. Inoltre, assumiamo che ogni nodo condivide una stessa idea di orientamento, ossia i nodi hanno un'idea di **destra** e **sinistra**.

Sotto queste assunzioni, il problema di leader election si può restringere ad un problema di **minimum finding (maximum finding)**, ossia trovare il nodo con l'identificatore minimo (massimo) e dichiararlo leader.

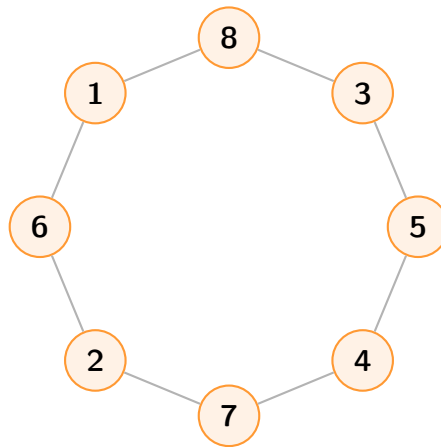


Figura 8.1: Rappresentazione di un anello a 8 nodi.

Osservazione. Se consideriamo un albero invece di un anello, basta eseguire la tecnica di saturazione per calcolare il minimo $id(x)$ ed eleggere tale nodo come leader.

8.1 All The Way

Consideriamo il primo protocollo, detto **all the way**, il quale consiste di far conoscere la propria etichetta a tutti gli altri nodi.

L'idea è la seguente: quando un nodo x si attiva (spontaneamente o dopo aver ricevuto un messaggio), inoltra un messaggio in direzione concorde a tutti (ad esempio a destra) con all'interno $id(x)$. Quando un nodo riceve un messaggio con l'id di qualcun'altro, invia il

proprio id se non lo ha ancora fatto, aggiora una variabile locale *min* che tiene traccia del id minimo ricevuto fin'ora e inoltra il messaggio.

Possiamo vedere il processo come segue: ogni entità crea un messaggio contenente il proprio id, e questo messaggio attraversa tutta la rete. Ogni entità, per via delle restrizioni di *TR*, verà entro un tempo finito tutti gli id di tutti gli agenti, incluso il valore minimo. Potrà quindi determinare se è il minimo o meno, e quindi, se entrare nello stato **leader** o **follower**.

La domanda che sorge è: quando un nodo capisce di aver visto gli id di tutti gli altri nodi? Il valore n non è noto, perciò è necessario trovare un modo per calcolare il valore n .

Il numero n è possibile calcolarlo inserendo un contatore all'interno del messaggio, che viene incrementato ad ogni nodo attraversato. Quando il messaggio torna al nodo di partenza, esso contiene il numero n di nodi presenti nella rete. Dunque, quando un nodo x riceve il suo messaggio da sinistra potrà ricavare il valore n e comportarsi in due modi:

- Se x ha ricevuto n messaggi (notiamo che viaggiano n messaggi, ciascuno contenente un id diverso), e $id(x)$ è il minimo, allora x si dichiara **leader**, altrimenti si dichiara **follower**.
- Se x ha ricevuto $k < n$ messaggi, allora dovrà aspettare di ricevere altri $n - k$ messaggi.

In questo modo ogni nodo è in grado di terminare localmente, e solo chi ha l'id minimo si dichiara leader.

Siano

- $S = \{\text{asleep}, \text{awake}, \text{follower}, \text{leader}\}$
- $S_{init} = \{\text{asleep}\}$
- $S_{final} = \{\text{follower}, \text{leader}\}$

8.1.1 Analisi

Il protocollo è corretto, in quanto l'assunzione di **TR** garantisce che ogni nodo prima o poi riceverà le informazioni riguardo le etichette di tutti quanti, dunque riuscirà a calcolare il minimo. Inoltre, la message complexity è

$$M[\text{All The Way}] = O(n^2),$$

in quanto su ciascun arco passano esattamente n messaggi di n etichette distinte, il numero degli archi è n , dunque n^2 . Possiamo inoltre calcolare la **bit complexity**, ossia il numero di bit scambiati. Si devono calcolare il numero di bit necessari per rappresentare $id(x)$ e il contatore. Il numero di bit necessari per rappresentare $id(x)$ è $\log_2 id(x)$ (oppure, $\log_2 n$ se consideriamo gli id come una permutazione di $\{1, 2, \dots, n\}$), mentre il numero di bit necessari per rappresentare il contatore è $\log n$, poichè il contatore può assumere al massimo il valore n . Dunque, ogni messaggio contiene $O(\log n)$ bit, e poichè passano $O(n^2)$ messaggi, la bit complexity è

$$B[\text{All The Way}] = O(n^2 \log n).$$

Algorithm 7 All The Way

Upon spontaneous awakening:

- 1: **if** state(x) = **asleep** **then**
- 2: INITIALIZE
- 3: state(x) $\leftarrow$ **awake**
- 4: **end if**

Upon receiving $msg = \langle \text{"Election"}, id, counter \rangle$ from left:

- 5: **if** state(x) = **asleep** **then**
- 6: INITIALIZE
- 7: **send** $\langle \text{"Election"}, id, counter + 1 \rangle$ to right
- 8: $min \leftarrow \min(id, min)$
- 9: $count \leftarrow count + 1$
- 10: state(x) $\leftarrow$ **awake**
- 11: **end if**

- 12: **if** state(x) = **awake** **then**
- 13: **if** $id \neq id(x)$ **then**
- 14: $min \leftarrow \min(id, min)$
- 15: $count \leftarrow count + 1$
- 16: **send** $\langle \text{"Election"}, id, counter + 1 \rangle$ to right
- 17: **if** known = **True** **then**
- 18: CHECK
- 19: **end if**
- 20: **else**
- 21: $size \leftarrow counter$
- 22: known $\leftarrow$ **True**
- 23: CHECK
- 24: **end if**
- 25: **end if**

- 26: **procedure** INITIALIZE
- 27: $count \leftarrow 1$
- 28: $min \leftarrow id(x)$
- 29: $size \leftarrow 1$
- 30: known $\leftarrow$ **False**
- 31: **send** $\langle \text{"Election"}, id(x), 1 \rangle$ to right
- 32: **end procedure**

- 33: **procedure** CHECK
- 34: **if** count = size **then**
- 35: **if** $min = id(x)$ **then**
- 36: state(x) $\leftarrow$ leader
- 37: **else**
- 38: state(x) $\leftarrow$ follower
- 39: **end if**
- 40: **end if**
- 41: **end procedure**

Infine, consideriamo l'ideal time complexity. Sia

$$R \cup \{\text{Synchronized Clock, Bounded Communication Delay}\}$$

in questo caso, $T[\text{All The Way}] = O(n)$. Il caso peggiore è quando si attiva spontaneamente un solo nodo x_1 al tempo t_0 . Inviando il messaggio al suo vicino x_2 , esso si attiverà al tempo t_2 , e così via fino al nodo x_n che si attiverà dopo $n - 1$ istanti.

8.2 As Far

Questo protocollo è una versione più efficiente di **All The Way**. Ogni nodo x blocca l'inoltro di tutti gli id che sono maggiori del proprio id, e inoltra solo quelli minori. Allora un messaggio con un certo id viaggia in avanti più a lungo che può.

In questo caso la terminazione locale dei nodi cambia. Infatti,

- Sia l il nodo con l'id minimo, allora l prima o poi riceverà un messaggio con il proprio id (il messaggio contenente il suo id sarà l'unico che non verrà mai bloccato), e a quel punto si dichiarerà leader, e invierà un messaggio di notifica in **broadcast** a tutti gli altri nodi, che si dichiareranno follower e termineranno localmente.
- Sia x un nodo con $id(x) > id(l)$, allora x riceverà un messaggio con un id minore del proprio, e quindi inoltrerà il messaggio, ma non riceverà mai un messaggio con il proprio id, dunque non si dichiarerà leader, ma aspetterà di ricevere un messaggio di notifica da parte del nodo leader, e una volta ricevuto, si dichiarerà follower e terminerà localmente.

Siano

- $S = \{\text{asleep, awake, follower, leader}\}$
- $S_{init} = \{\text{asleep}\}$
- $S_{final} = \{\text{follower, leader}\}$

Algorithm 8 All The Way

Upon spontaneous awakening:

```
1: if state(x) = asleep then  
2:     INITIALIZE  
3:     state(x)  $\leftarrow$  awake  
4: end if
```

Upon receiving $msg = \langle \text{"Election"}, id \rangle$ from left:

```
5: if state(x) = asleep then  
6:     INITIALIZE  
7:     if  $id < min$  then  
8:         send  $\langle \text{"Election"}, id \rangle$  to right  
9:          $min \leftarrow id$   
10:     end if  
11:     state(x)  $\leftarrow$  awake  
12: end if
```

```
13: if state(x) = awake then  
14:     if  $id < min$  then  
15:         send  $\langle \text{"Election"}, id \rangle$  to right  
16:          $min \leftarrow id$   
17:     else  
18:         if  $id = min$  then  
19:             state(x)  $\leftarrow$  leader  
20:             send  $\langle \text{"Leader"} \rangle$  to right  
21:         end if  
22:     end if  
23: end if
```

Upon receiving $msg = \langle \text{"Leader"} \rangle$ from left:

```
24: if state(x) = awake then  
25:     state(x)  $\leftarrow$  follower  
26:     send  $\langle \text{"Leader"} \rangle$  to right  
27: end if
```

```
28: procedure INITIALIZE  
29:      $min \leftarrow id(x)$   
30:     send  $\langle \text{"Election"}, id(x) \rangle$  to right  
31: end procedure
```

8.2.1 Analisi

L'esatto numero di messaggi scambiati dipende dalla disposizione degli id e dal senso di invio. Consideriamo il costo causato dal messaggio **Election** di un nodo x . Tale messaggio viaggerà attraverso l'anello sino a quando non viene ricevuto da un nodo che ha ricevuto in precedenza un id minore, oppure sino a quando non raggiunge il proprio mittente.

Consideriamo il caso peggiore, siano $id_1 < id_2 < \dots < id_n$ gli id dei nodi nell'anello e supponiamo che l'invio avvenga in senso orario (verso destra). L' id_1 attraverserà sempre tutto l'anello avendo quindi un costo di n messaggi, l' id_2 attraverserà $n - 1$ nodi, venendo bloccato da id_1 , l' id_3 attraverserà $n - 2$ nodi, e così via. Dunque, il costo totale è

$$M[\text{As Far}] = 2n + (n - 1) + (n - 2) + \dots + 1 = n + \sum_{i=1}^n i = n + \frac{n(n + 1)}{2} = O(n^2).$$

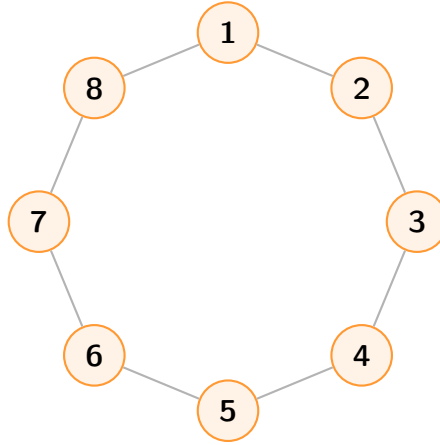


Figura 8.2: Worst case per il protocollo **As Far**: gli ID sono disposti in ordine crescente in senso orario.

Il caso migliore si ottiene quando il nodo con etichetta minima inizia il protocollo per primo e la sua etichetta riesce a fare il giro completa della rete prima che le altre vengano inoltrate al proprio vicino. In particolare, il messaggio con etichetta minima viene inoltrato n volte, mentre ogni messaggio con etichetta maggiore viene inoltrato una sola volta, dunque

$$M[\text{As Far}] = n + (n - 1) = O(n).$$

8.3 Stage Technique

Osserviamo che sia nel protocollo **As Far** che nel protocollo **All The Way**, le etichette viaggiano lungo l'anello in maniera incontrollata. In particolare, nel protocollo **As Far** viaggiano finché non vengono filtrati da un nodo con etichetta minore, mentre nel protocollo **All The Way** viaggiano finché non fanno il giro completo dell'anello. L'idea sarebbe quindi quella di **controllare** la trasmissione di un'etichetta ad una distanza **limitata**, e di avere dei **feedback** per eventualmente bloccare o estendere la trasmissione.

Più precisamente un nodo attivo x esegue il protocollo in **stages**, in cui al i -esimo stage, x invia la sua etichetta $id(x)$ a destra e sinistra fino a un massimo di distanza di 2^{i-1} . Così facendo l'etichetta avrà ricoperto un numero di nodi pari a 2^i , ovvero 2^{i-1} a destra e 2^{i-1} a sinistra.

Quando un nodo y riceve l'etichetta di x la inoltrerà solo se $id(y) > id(x)$. Nel caso in cui y inoltrerà $id(x)$ allora vorrà dire che certamente y non sarà mai eletto leader, e quindi passerà in uno stato **defeated** in cui potrà solamente inoltrare messaggi di altri nodi.

Se l'etichetta $id(x)$ arriva a distanza 2^{i-1} , allora x ha completato il i -esimo stage e verrà rispedita indietro come messaggio di **feedback**. Quindi se a x arriva la sua etichetta da **entrambi** i lati, vorrà dire che tutti i nodi a distanza 2^{i-1} da lui avranno etichette maggiori, e quindi dovrà inoltrare la richiesta più lontano. In questo caso diciamo che x ha sopravvissuto allo stage i e quindi passerà allo stage $i + 1$.

Notiamo che se x non riceve alcun feedback da **almeno** un lato, allora vorrà dire che esiste un nodo z con $id(z) < id(x)$ entro 2^{i-1} passi da x , e quindi x non potrà essere eletto leader, dunque passerà in stato **defeated**.

Infine, quando un nodo l riceverà da destra (o sinistra) il messaggio che aveva inviato a sinistra (o destra), allora l si dichiarerà leader, e invierà un messaggio di notifica in **broadcast** a tutti gli altri nodi, che si dichiareranno follower e termineranno localmente.

Osservazione. Durante l'esecuzione del protocollo, è possibile che diversi nodi si trovino in diversi stages. Ci si chiede quindi cosa succede se un nodo viene "sconfitto" da un messaggio dello stage k mentre lui si trova nello stage $i < k$. In questo caso, il nodo sconfitto non potrà più essere eletto leader, dunque passerà in stato **defeated**, e potrà solamente inoltrare messaggi di altri nodi. (NB, il minimo id vince sempre.)

Siano

- $S = \{\text{asleep}, \text{candidate}, \text{defeated}, \text{follower}, \text{leader}\}$
- $S_{init} = \{\text{asleep}\}$
- $S_{final} = \{\text{follower}, \text{leader}\}$

Algorithm 9 Stage Technique

Upon spontaneous awakening:

```
1: if state(x) = asleep then  
2:   INITIALIZE  
3:   state(x)  $\leftarrow$  candidate  
4: end if
```

Upon receiving $msg = \langle \text{"Forth"}, id^*, stage^*, limit^* \rangle$:

```
5: if state(x) = asleep then  
6:   if  $id^* < id(x)$  then  
7:     PROCESS-MESSAGE  
8:     state(x)  $\leftarrow$  defeated  
9:   else  
10:    INITIALIZE  
11:    state(x)  $\leftarrow$  candidate  
12:   end if  
13: end if  
  
14: if state(x) = candidate then  
15:   if  $id^* < id(x)$  then  
16:     PROCESS-MESSAGE  
17:     state(x)  $\leftarrow$  defeated  
18:   else  
19:     if  $id^* = id(x)$  then  
20:       NOTIFY  
21:     end if  
22:   end if  
23: end if
```

Upon receiving $msg = \langle \text{"Back"}, id^* \rangle$ from left:

```
24: if state(x) = candidate then  
25:   if  $id^* = id(x)$  then  
26:     CHECK  
27:   end if  
28: end if
```

Upon receiving $msg = \langle \text{"Notify"} \rangle$ from left:

```
29: if state(x) = candidate then  
30:   send  $\langle \text{"Notify"} \rangle$  to other  
31:   state(x)  $\leftarrow$  follower  
32: end if
```

Upon receiving $msg = \langle * \rangle$:

```
33: if state(x) = defeated then  
34:   send  $msg$  to other  
35:   if  $msg = \langle \text{"Notify"} \rangle$  then  
36:     state(x)  $\leftarrow$  follower  
37:   end if  
38: end if
```

Algorithm 10 Stage Technique

```
39: procedure INITIALIZE
40:   stage  $\leftarrow 1$ 
41:   limit  $\leftarrow \text{dis}(\text{stage})$ 
42:   count  $\leftarrow 0$ 
43:   send  $\langle \text{"Forth"}, \text{id}(x), \text{stage}, \text{limit} \rangle$  to  $N(x)$ 
44: end procedure

45: procedure PROCESS-MESSAGE
46:   limit*  $\leftarrow \text{limit}^* - 1$ 
47:   if limit* = 0 then
48:     send  $\langle \text{"Back"}, \text{id}^*, \text{stage}^* \rangle$  to sender
49:   else
50:     send  $\langle \text{"Forth"}, \text{id}^*, \text{stage}^*, \text{limit}^* \rangle$  to other
51:   end if
52: end procedure

53: procedure CHECK
54:   count  $\leftarrow \text{count} + 1$ 
55:   if count = 1 then
56:     count  $\leftarrow 0$ 
57:     stage  $\leftarrow \text{stage} + 1$ 
58:     limit  $\leftarrow \text{dis}(\text{stage})$ 
59:     send  $\langle \text{"Forth"}, \text{id}(x), \text{stage}, \text{limit} \rangle$  to  $N(x)$ 
60:   end if
61: end procedure

62: procedure NOTIFY
63:   send  $\langle \text{"Notify"} \rangle$  to right
64:   state(x)  $\leftarrow \text{leader}$ 
65: end procedure
```

8.3.1 Analisi

La correttezza del protocollo deriva dalla sua definizione. Infatti, solamente l'etichetta minima sarà in grado di sopravvivere a tutti gli stages (e quindi a fare un giro completo), e quindi di dichiararsi leader, mentre tutte le altre etichette saranno bloccate da un nodo con etichetta minore, e quindi si dichiareranno follower.

Analizziamo ora la message complexity del protocollo. Considerando che il ritardo dei messaggi è **impredicibile** si osserverà che ad un tempo t non solo ci saranno nodi in stati differenti, ma anche che ci saranno nodi *candidati* in **stages differenti**. Infatti, non essendoci una **Message Ordering**, può capitare che dei messaggi sorpassino degli altri. Perciò, faremo un'analisi procedendo per **logical stages**, ossia, al i -esimo stage, considereremo solamente i messaggi relativi allo stage i , e faremo un'analisi indipendente per ogni stage (non consideriamo che differenti nodi possono eseguire uno stesso stage in tempi differenti, come abbiamo visto, se un nodo nello stage $i - 1$ riceve un messaggio da un nodo nello stage i , vince sempre il nodo con l'etichetta minore (approccio greedy)).

Teorema 8.3.1 (Message Complexity). *La message complexity del protocollo Stage Technique è*

$$M[\text{Stage Technique}] = O(n \log n).$$

Dimostrazione. Sia $i > 1$, definiamo n_i il numero di nodi che iniziano lo stage i , ovvero quelli sopravvissuti allo stage $i - 1$. Osserviamo che

$$n_i \leq \frac{n}{2^{i-2} + 1},$$

in quanto, se un nodo sopravvive allo stage $i - 1$, allora tutti i nodi entro una distanza di 2^{i-2} da lui hanno etichetta maggiore, dunque, in totale, ci sono al massimo $\frac{n}{2^{i-2}}$ nodi che possono sopravvivere allo stage i (un solo nodo su 2^{i-2} può sopravvivere).

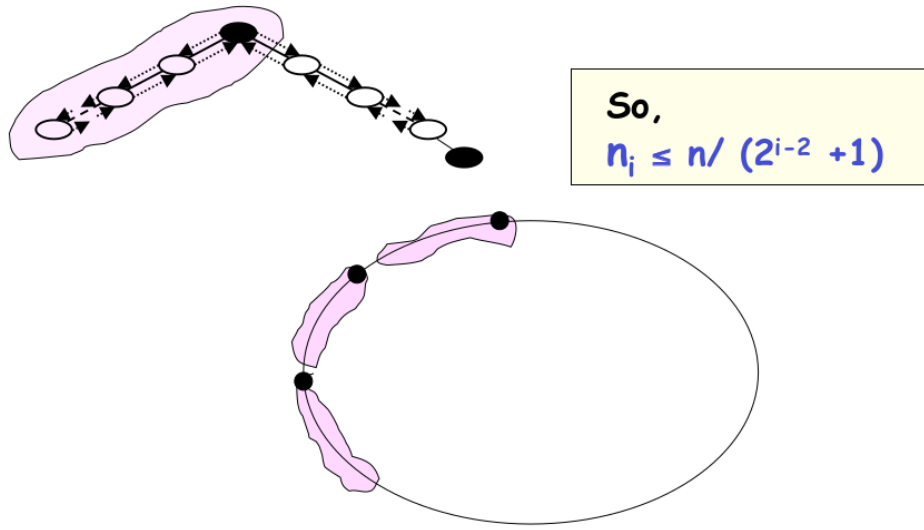


Figura 8.3: Nodi che sopravvivono allo stage i .

Non ci può essere più di un sopravvissuto ogni 2^{i-2} nodi in quanto se ce ne fossero di più, per esempio x e y , vorrebbe dire che x ed y si sono scambiati le etichette senza però che uno dei due venisse "sconfitto", e ciò implicherebbe che $id(x) < id(y) < id(x)$.

Contiamo ora il numero di messaggi scambiati durante lo stage i . Abbiamo due tipologie di messaggi, quelli di tipo **Forth** e quelli di tipo **Back**.

FORTH: Ogni nodo candidato x inoltra la sua etichetta ad **al più** $2 \cdot 2^{i-1} = 2^i$ nodi, per un totale di $n_i \cdot 2^i$

BACK: Poiché nella fase i sopravviveranno n_{i+1} candidati, per ognuno verranno necessariamente trasmessi altri $2 \cdot 2^{i-1} = 2^i$ messaggi di feedback, per un totale di $n_{i+1} \cdot 2^i$.

Infine, rimangono i nodi **defeated**, che sono $n_i - n_{i+1}$. Questi riceveranno al massimo un messaggio di tipo **Back**, per un massimo di 2^{i-1} ulteriori messaggi, dunque, in totale, **al più** $2^{i-1} \cdot (n_i - n_{i+1})$ messaggi.

Perciò, complessivamente, nella fase $i > 1$,

$$\begin{aligned}
M[\text{stage}_i] &= 2n_i 2^{i-1} + 2n_{i+1} 2^{i-1} + (n_i - n_{i+1}) 2^{i-1} = (3n_i + n_{i+1}) 2^{i-1} \\
&\leq \left(3 \left\lfloor \frac{n}{2^{i-2} + 1} \right\rfloor + \left\lfloor \frac{n}{2^{i-2} + 1} \right\rfloor \right) 2^{i-1} \\
&< \left(3 \frac{n}{2^{i-2} + 1} + \frac{n}{2^{i-2} + 1} \right) 2^{i-1} \\
&< \left(4 \frac{n}{2^{i-2}} \right) 2^{i-1} \\
&= 6n + n = 7n.
\end{aligned}$$

Osservazione. Il numero di messaggi scambiati durante lo stage 1 è diverso, poichè il caso peggiore si ha quando tutti i nodi sono iniziatori. In questo caso

$$n_2 \leq \frac{n}{2^0 + 1} = \frac{n}{2}$$

I nodi che sopravvivono allo stage 1 pagano individualmente 2 per **Forth** e 2 per **Back**, mentre i nodi che vengono sconfitti pagano individualmente 2 per **Forth** 1 per **Back**, in totale,

$$M[\text{stage}_1] = 4n_2 + 3n - 3n - 3n_2 = n_2 + 3n < 4n.$$

Numero di stage. Rimane solo da dimostrare che il numero di stage è $O(\log n)$. Sia i^* lo stage in cui un nodo si dichiara leader, ossia quando riceverà da un lato il messaggio che ha inviato dal lato opposto, allora

$$i^* = \min\{i \in \mathbb{N} : 2^{i-1} \geq n\} \iff i^* = O(\log n)$$

Dunque il numero necessari di stage è $O(\log n)$, e poichè in ogni stage vengono scambiati al più $7n$ messaggi, la message complexity totale è

$$M[\text{Stage Technique}] = \sum_{i=1}^{\log n} M[\text{stage}_i] + M[\text{stage}_1] = O(n \log n).$$

□

8.4 Mesh Architecture

Definizione 8.4.1 (Mesh). Una mesh è un grafo $\mathcal{M} = (V, E)$ dove:

$$\begin{aligned} V &= \{x_{i,j} : 1 \leq i \leq a, 1 \leq j \leq b\}, & |V| &= n = ab \\ E &= \{(x_{i,j}, x_{i+1,j}) : 1 \leq i < a, 1 \leq j \leq b\} \\ &\cup \{(x_{i,j}, x_{i,j+1}) : 1 \leq i \leq a, 1 \leq j < b\}, & |E| &= m = 2ab - a - b. \end{aligned}$$

Una mesh è una griglia rettangolare di dimensioni $a \times b$ dove ogni nodo è collegato ai suoi vicini orizzontali e verticali.

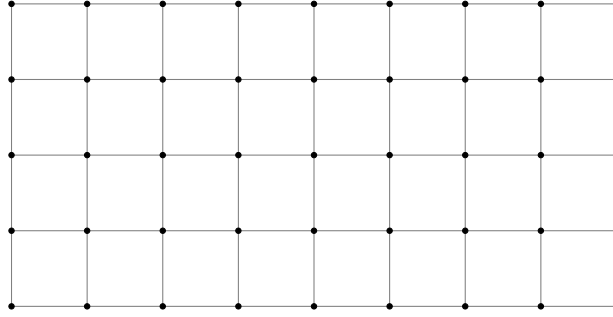


Figura 8.4: Rappresentazione di una mesh 9×5 .

In una mesh, possiamo distinguere tre tipi di nodi (è una topologia **asimmetrica**):

- **Corner nodes**: sono i nodi che si trovano agli angoli della griglia, e hanno grado 2.
- **Edge nodes**: sono i nodi che si trovano sui bordi della griglia, ma non agli angoli, e hanno grado 3 e ve ne sono $2(a + b) - 4$.
- **Inner nodes**: sono i nodi che si trovano all'interno della griglia, e hanno grado 4 e ve ne sono $(a - 2)(b - 2)$.

8.4.1 Protocollo

Consideriamo il sottografo indotto dai nodi **corner** e dai nodi **edge**, che formano un anello. Eseguiamo su tale anello un protocollo di leader election per eleggere un leader tra i nodi **corner**. Essi saranno gli unici candidati, mentre i nodi **edge** avranno solo il ruolo di inoltrare. Una volta eletto il leader, esso esegue una **broadcast** a tutta la rete, e ogni nodo si dichiara follower e termina localmente.

Il protocollo consiste di tre fasi:

1. **Wake Up**: Ogni initiator (k) invia un messaggio di **wake-up** a tutti i suoi vicini. Ogni nodo non initiator che riceve il messaggio di **wake-up**, lo invia agli altri vicini, in totale si scambiano

$$M[\text{Wake Up}] = 3n + k = O(n).$$

2. **Election on the Ring**: Poichè i nodi sanno di essere o edge o corner, possono eseguire un protocollo di leader election sull'anello formato da questi nodi. I nodi corner sono gli unici candidati, che inviano i propri id, mentre i nodi edge si limitano ad inoltrare i messaggi. Quindi, un nodo sa che ci sono esattamente 4 candidati, e

una volta che ha ricevuto 4 id diversi può calcolare il minimo e dichiararsi o leader o follower e terminare localmente. La message complexity è:

$$M[\text{Election on the Ring}] = O(a + b) = O(\sqrt{n}),$$

poichè su ogni link dell'anello passano al più 4 messaggi, e l'anello è formato da $2(a + b) - 4$ nodi.

3. **Broadcast:** Il nodo leader esegue una **broadcast** a tutta la rete, e ogni nodo si dichiara follower e termina localmente. La message complexity è

$$M[\text{Broadcast}] = O(n),$$

8.5 Randomized Algorithm

Abbiamo visto che il problema della leader election non è risolvibile in modo deterministico in una rete in cui i nodi sono anonimi, ossia non esiste una etichettamento univoco dei nodi.

Consideriamo ora un anello $G = (V, E)$ di dimensione n , dove i nodi sono **anonimi**, e conoscono la topologia della rete e il numero di nodi n . In questo caso, è possibile risolvere il problema della leader election in modo **randomizzato**.

Ciascun nodo $x \in V$ sceglie, indipendentemente dagli altri nodi, la sua etichetta

$$id(x) \xleftarrow{u.a.r.} [m] = \{1, \dots, m\},$$

dove m è un parametro che dipende da n e che verrà scelto in modo opportuno, sperando che non esistano due nodi con la stessa etichetta. A questo punto, i nodi eseguono uno dei protocolli visti in precedenza (**All The Way, As Far, Stage Technique**) per eleggere un leader tra i nodi che hanno scelto l'etichetta minima.

Formalmente, il protocollo $\mathcal{RL}(G, n; m)$ è il seguente (m è un parametro che dipende da n e che verrà scelto in modo opportuno):

1. **Wake-up:** Viene eseguito un protocollo di wake-up, in modo che tutti i nodi si attivino.
2. **Random ID Generation:** Ogni nodo x sceglie, indipendentemente dagli altri nodi, la sua etichetta $id(x)$ in modo uniforme a caso da $[m]$.

$$id(x) \xleftarrow{u.a.r.} [m] = \{1, \dots, m\},$$

3. **Leader Election:** I nodi eseguono uno dei protocolli *deterministici* di leader election visti in precedenza, per eleggere un leader tra i nodi che hanno scelto l'etichetta minima.

8.5.1 Analisi

Analizziamo ora la correttezza del protocollo, la message complexity ovviamente dipende da quale protocollo deterministico viene scelto, che abbiamo già analizzato, dunque ci concentriamo sulla probabilità di successo del protocollo.

Dobbiamo fare un'analisi probabilistica del protocollo. Innanzitutto definiamo lo spazio di probabilità, che è dato da tutte le possibili assegnazioni di etichette ai nodi, dunque

$$\Omega = [m]^n = \{(id(x_1), id(x_2), \dots, id(x_n)) : id(x_i) \in [m], 1 \leq i \leq n\}.$$

Osserviamo che un **evento sufficiente** (ma **non necessario**) affinché il protocollo termini **correttamente** è:

$$\mathcal{B} = \text{"non esistono due nodi differenti } u, w \in V, u \neq w : id(u) = id(w)\text{"}$$

L'evento $\mathcal{B}$ è sufficiente perché se non esistono due nodi con la stessa etichetta, allora esiste un nodo con etichetta minima, e tale nodo sarà l'unico candidato, dunque verrà eletto leader, e tutti gli altri nodi si dichiareranno follower e termineranno localmente. Tuttavia, $\mathcal{B}$ non è necessario perché è possibile che esistano due nodi $u, v \in V$ con $u \neq v$ e $id(u) = id(v)$ ma che esista un nodo w con $id(w) < id(u) = id(v)$, e quindi w sarà l'unico candidato, e verrà eletto leader, e tutti gli altri nodi si dichiareranno follower e termineranno localmente.

Osserviamo che possiamo modellare il problema come un problema di **urne e palline** (*Balls-into-Bins Process*), in cui abbiamo n palline (i nodi) e m urne (le possibili etichette), e ogni pallina viene lanciata in modo uniforme a caso in una delle m urne. L'evento $\mathcal{B}$ si verifica se e solo se tutte le palline finiscono in urne diverse, ossia non esistono due palline che finiscono nella stessa urna.

Calcoliamo ora la probabilità che si verifichi l'evento $\mathcal{B}$.

$$\mathcal{P}(\mathcal{B}) = 1 - \mathcal{P}(\overline{\mathcal{B}}) = 1 - \mathcal{P}(\text{"esistono due nodi } u, w \in V, u \neq w : id(u) = id(w)\text{"})$$

Per ogni $i \in [m]$, definiamo l'evento

$$\overline{\mathcal{B}}_i = \text{"esistono due nodi } u, w \in V, u \neq w : id(u) = id(w) = i\text{"}.$$

Possiamo vedere l'evento $\overline{\mathcal{B}}_i$ come "almeno due palline finiscono nell'urna i ", quindi,

Tale evento può essere modellato come l'unione di v.a. binomiali.

$$\begin{aligned} \mathcal{P}(\overline{\mathcal{B}}_i) &= \mathcal{P}\left(\bigcup_{k=2}^n \{\text{esistono } k \text{ palline nell'urna } i\}\right) \\ \text{unione di eventi disgiunti} &= \sum_{k=2}^n \mathcal{P}(\text{esistono } k \text{ palline nell'urna } i) \\ &= \sum_{k=2}^n \binom{n}{k} \left(\frac{1}{m}\right)^k \left(1 - \frac{1}{m}\right)^{n-k} \leq \sum_{k=2}^n \binom{n}{k} \left(\frac{1}{m}\right)^k \\ \text{Stirling's approx} &\leq \sum_{k=2}^n \left(\frac{ne}{k} \cdot \frac{1}{m}\right)^k \leq O\left(\frac{n^2}{m^2}\right) + \sum_{k=3}^n \left(\frac{ne}{k} \cdot \frac{1}{m}\right)^k \\ &\leq O\left(\frac{n^2}{m^2} + \frac{n^4}{m^3}\right) \end{aligned}$$

Adesso, possiamo calcolare la probabilità che si verifichi l'evento $\overline{\mathcal{B}}$, ossia che esistano due nodi con la stessa etichetta, utilizzando l'unione di eventi:

$$\begin{aligned}\mathcal{P}(\overline{\mathcal{B}}) &= \mathcal{P}\left(\bigcup_{i=1}^m \overline{\mathcal{B}}_i\right) \\ \text{union bound} &\leq \sum_{i=1}^m \mathcal{P}(\overline{\mathcal{B}}_i) \\ &\leq m \cdot O\left(\frac{n^2}{m^2} + \frac{n^4}{m^3}\right) = O\left(\frac{n^2}{m} + \frac{n^4}{m^2}\right)\end{aligned}$$

Ponendo ora $m = n^3$, avremo che tale probabilità sarà al più

$$\mathcal{P}(\overline{\mathcal{B}}) = O\left(\frac{n^2}{n^3} + \frac{n^4}{n^6}\right) \in O\left(\frac{1}{n}\right).$$

Possiamo quindi concludere che per $m \geq n^3$, l'esecuzione del protocollo $\mathcal{RL}(G, n; m)$ porterà la rete in una configurazione finale accettabile con **alta probabilità (w.h.p.)**

$$\mathcal{P}(\overline{\mathcal{B}}) = 1 - O\left(\frac{1}{n}\right)$$

Capitolo 9

Graph Coloring

9.1 Deterministic Distributed Graph Coloring

9.1.1 Basic Color Reduction Procedure - BCP

9.1.2 Khun-Wattenhofen Procedure

9.2 Randomized Distributed Graph Coloring

9.2.1 Rand- 2Δ

9.2.2 Rand- $\Delta+$